SIMULATORPAYMENT

Manual de Usuario y Referencia Técnica

Simulador transaccional Base24 · Java 17 · JavaFX · H2 · Netty

Guía funcional de operación, configuración, pruebas, mensajería ISO8583, seguridad y arquitectura interna.

Versión del documento: 1.0
Fecha: 30 de septiembre de 2026
Alcance: interfaz v1.3 y esquema H2 versión 6

Control del documento

Elemento	Detalle
Documento	Manual de Usuario y Referencia Técnica de SimulatorPayment
Audiencia	Operadores, analistas QA, desarrolladores, integradores y administradores de ambientes Base24
Propósito	Explicar para qué sirve cada módulo, cómo se configura, cómo se opera y cómo se integra técnicamente
Versión cubierta	Interfaz v1.3; H2/Flyway V6; Java 17+; módulos Maven actuales
Convenciones	Los nombres entre comillas corresponden a textos reales de la interfaz. Los ejemplos deben adaptarse a cada instalaci

Cómo usar este manual

La primera parte está orientada al operador: instalación, navegación, configuración y uso paso a paso de cada pantalla. La segunda explica la arquitectura, los formatos ISO8583, el transporte, la persistencia y la extensión del producto. Las capturas se generaron sobre una instancia local aislada; los datos, estados y rutas pueden diferir de los de producción.

Importante. SimulatorPayment es una herramienta de laboratorio. Antes de conectarla a una red real, valide dialecto, framing, TLS, correlación, seguridad PCI, puertos y reglas con la especificación del host.

Índice de contenido

Parte	Contenido
Fundamentos	1. Visión general · 2. Instalación y navegación · 3. Dashboard
Transacciones	4. Adquirencia · 5. Emisión Cliente/Server y reglas
Mensajería	6. Parser · 7. Formatos/dialectos · 8. Framing · 9. Cripto/HSM
Pruebas	10. Regresión · 11. Grabación/Replay · 12. Stress
Operación	13. Base24 · 14. H2 · 15. Configuración · 16. Logs
Referencia técnica	17. Seguridad · 18. Headless · 19. Arquitectura · 20. Escenarios
Anexos	Checklists · Limitaciones · Glosario · Controles UI · Backend · Evidencia

El PDF incluye marcadores navegables para todos los capítulos y subcapítulos.

1. Visión general y arquitectura

SimulatorPayment es una aplicación de escritorio para simular, observar y automatizar intercambios transaccionales contra ACI Base24 u otros hosts ISO8583. Reúne adquirencia, emisión, parser, formatos, HSM de software, regresión, grabación/replay, stress, estadísticas y operaciones remotas en una única interfaz.

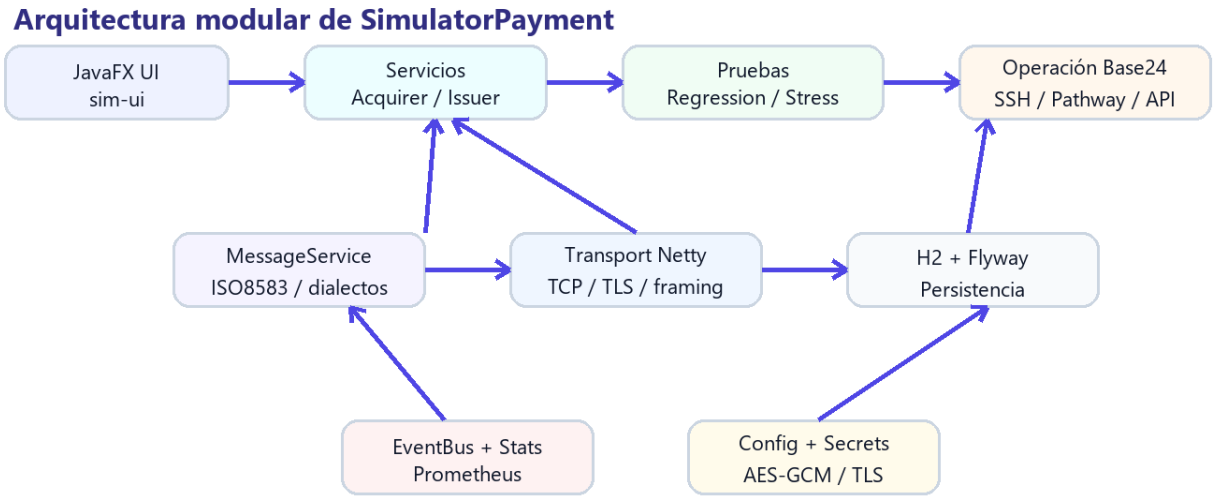


Figura 1. Arquitectura lógica: la UI cablea servicios transaccionales, transporte, mensajería, persistencia y operación remota.

Mapa funcional

Área	Módulos	Objetivo
Monitoreo	Dashboard, Logs	Observar volumen, estados, TPS, latencias y diagnóstico de la aplicación.
Transacciones	Adquirencia, Emisión	Enviar requests y simular hosts que reciben y responden mensajes.
Mensajería	Parser, Formatos, Cripto/HSM	Construir, inspeccionar, codificar, validar y proteger mensajes ISO8583.
Pruebas	Regresión, Grabación/Replay, Stress	Automatizar validaciones funcionales, reproducir tráfico y medir capacidad.
Base24	Guardian, Pathway	Ejecutar consultas, ABM y comandos remotos controlados.
Sistema	Base de datos, Configuración, Logs	Administrar H2, ambientes, secretos, TLS y diagnóstico.

Principios operativos

- La aplicación es monousuario y abre directamente en el Dashboard; no hay login ni roles.
- La contraseña maestra se solicita sólo cuando una acción necesita descifrar un secreto `ENC(...)`.
- El JSON es la representación de trabajo; el dialecto lo transforma en bytes ISO8583 y el framing delimita esos bytes en TCP.
- Los datos sensibles se enmascaran antes de persistir o exportar; el hex crudo se muestra sólo bajo demanda.
- La actividad transaccional alimenta el EventBus, las estadísticas, el audit log y, si corresponde, las grabaciones.

2. Instalación, arranque y navegación

La distribución incluye un JAR ejecutable y scripts Windows. El requisito mínimo es JDK 17; Maven 3.8+ sólo es necesario para compilar. La aplicación usa por defecto `~/.simulator/config`, `~/.simulator/data` y `~/.simulator/logs`.

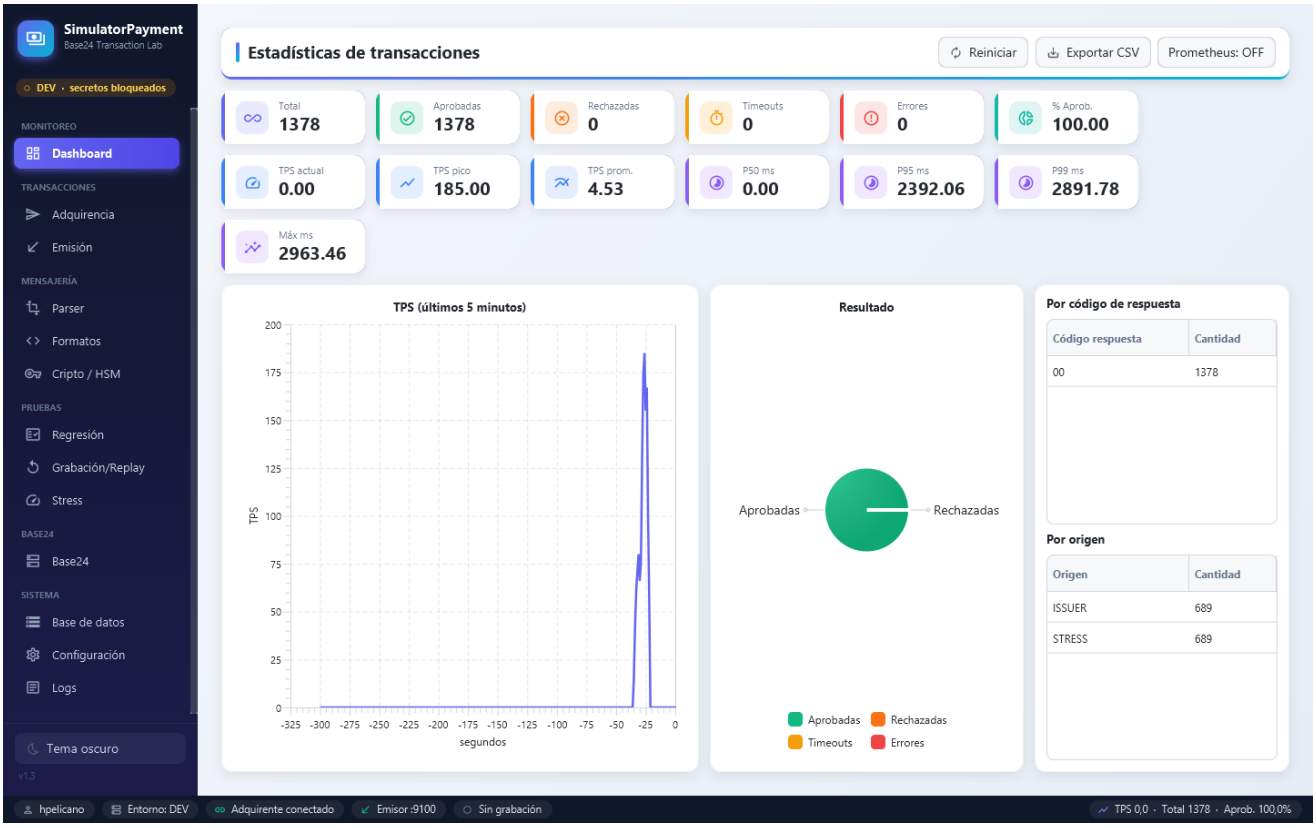


Figura 2. Ventana principal: navegación lateral agrupada, Dashboard y barra de estado.

Ejecución

```
run.bat
java -jar sim-app\target\simulator-payment.jar

# Directorios alternativos
java -Dsimulator.config=C:\lab\config ^
     -Dsimulator.data=C:\lab\data ^
     -Dsimulator.logs=C:\lab\logs ^
     -jar sim-app\target\simulator-payment.jar
```

Navegación y barra de estado

- La barra lateral agrupa las vistas en Monitoreo, Transacciones, Mensajería, Pruebas, Base24 y Sistema.
- El badge superior muestra el ambiente activo y si los secretos están bloqueados.
- La barra inferior informa usuario de sistema, ambiente, estado de Adquirencia, estado del Emisor, grabación, TPS, total y porcentaje aprobado.
- El botón “Tema claro/Tema oscuro” persiste la preferencia en `user.json`.
- Las vistas se adaptan mediante scroll; el tamaño preferido es 1440×900 y el mínimo 900×600.

Primer circuito local recomendado

Paso	Acción	Resultado esperado
1	Emisión → Modo Server → Full-duplex → Iniciar en 9100	El emisor queda escuchando.
2	Adquirencia → TCP → LEN2_BINARY → Conectar	Estado “Conectado: tcp://127.0.0.1:9100”.
3	Elegir tabla H2, suite/dialect y registro; presionar Send	Respuesta 0210/DE39 según la regla del emisor.
4	Revisar Respuesta, Validación, Hex y Trace respuesta	Evidencia funcional y técnica del intercambio.
5	Dashboard y Emisión → Log de mensajes	Contadores, latencia y trazabilidad.

3. Dashboard — monitoreo en tiempo real

El Dashboard resume la salud transaccional sin reemplazar al log técnico. Se actualiza cada segundo a partir de `StatisticsCollector`, que consume eventos de Adquirencia, Emisión, Regresión, Replay y Stress.

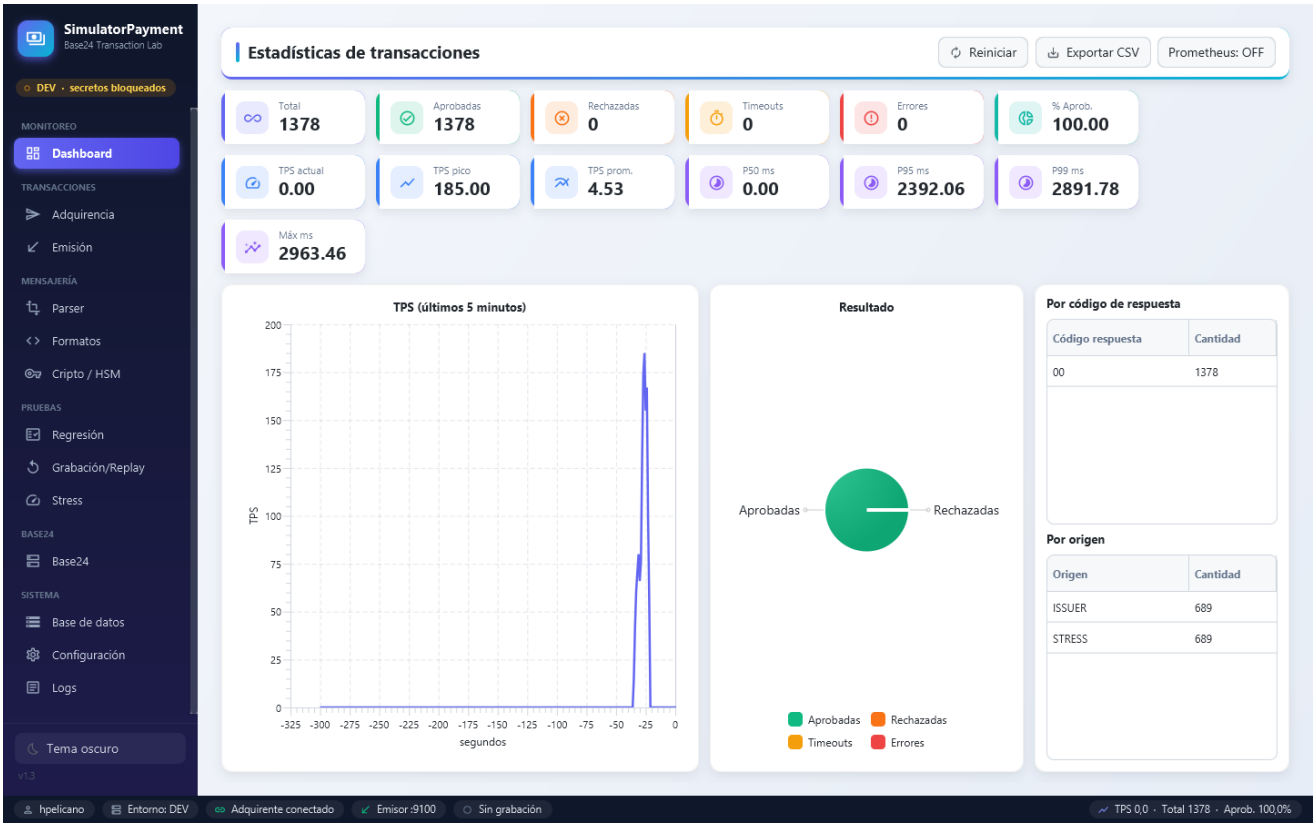


Figura 3. Dashboard con KPIs y gráficos de actividad.

Indicadores

Indicador	Significado
Aprobadas / Rechazadas	Clasificación por response code. RC 00, 08, 10, 11, 16, 000 y 001 se consideran aprobados.
Timeouts / Errores	Ausencia de respuesta dentro del timeout o fallas de conexión, codificación o ejecución.
TPS	Tasa actual, promedio y pico; útil para una lectura rápida durante stress.
Latencia	Promedio y percentiles p50, p95 y p99 calculados con HdrHistogram.
Series	TPS en el tiempo, resultados por outcome y response codes más frecuentes.

Acciones

- "Reiniciar" borra los acumuladores y comienza una nueva ventana estadística.
- "Exportar CSV" guarda la instantánea para análisis externo.
- El toggle de Prometheus inicia o detiene el endpoint HTTP en el puerto configurado (por defecto 9464).
- El endpoint expone contadores por source/outcome/RC, histograma de latencia, TPS, approval rate y gauges de conexiones.

Consejo. Para una prueba reproducible, reinicie las estadísticas antes de comenzar, documente la hora de inicio y exporte el CSV al finalizar.

4. Adquirencia — envío de transacciones

Adquirencia actúa como cliente de Base24 por TCP o API. Carga requests persistidos en H2, permite editarlos, enviarlos, validar la respuesta, generar reversos y guardar nuevas variantes.

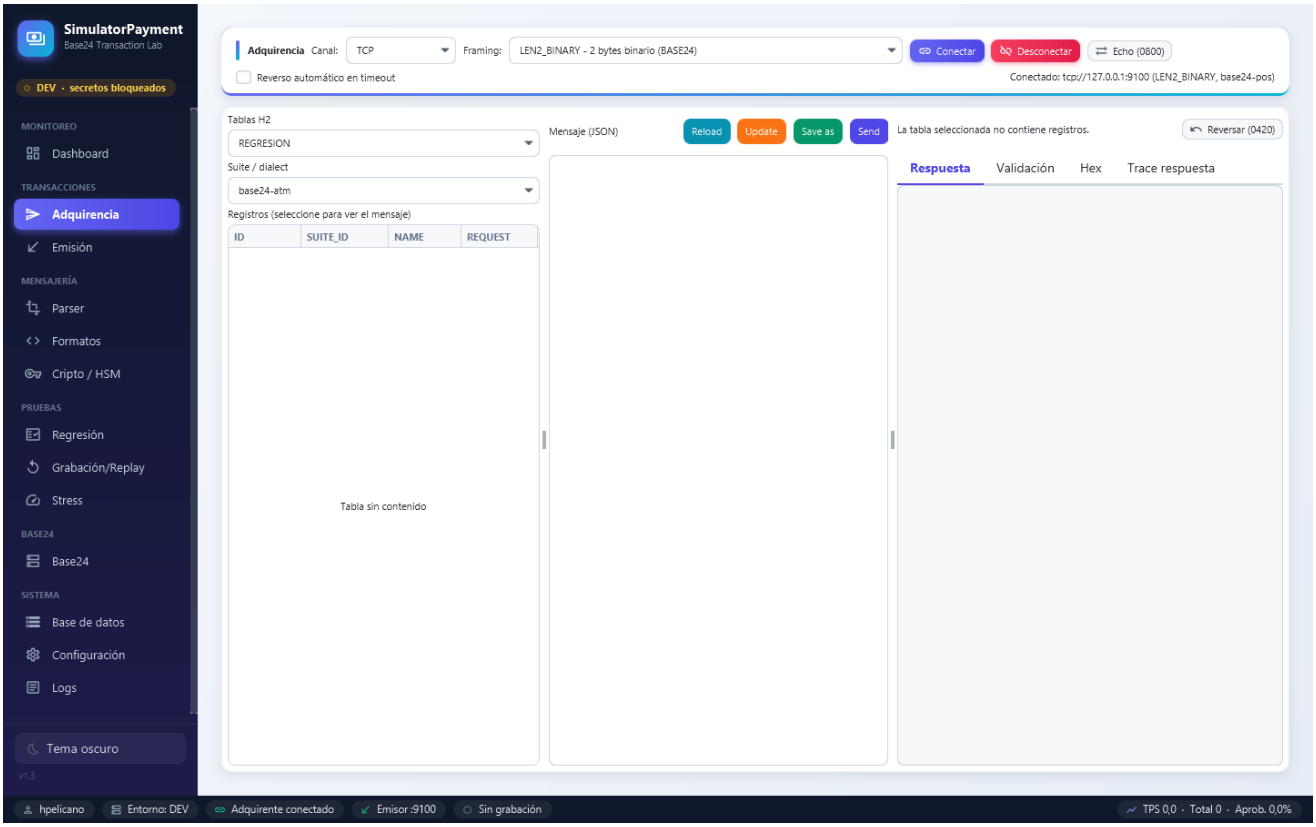


Figura 4. Adquirencia: conexión, catálogo H2, editor JSON y panel de respuesta.

Flujo de una transacción de Adquirencia

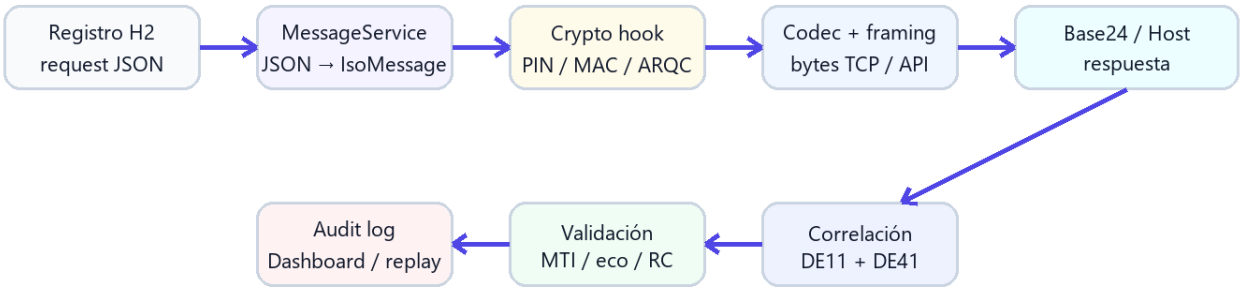


Figura 5. Recorrido del request desde H2 hasta la validación y auditoría.

Conexión

- Canal "TCP": usa `env.acquirer` (host, puerto, dialecto, timeouts, TLS y correlación).
- Canal "API": usa `env.api`; el selector de framing queda deshabilitado.
- "Conectar" crea el canal; "Desconectar" lo cierra. "Echo (0800)" emite una gestión de red con código 301.
- "Reverso automático en timeout" envía un 0420 cuando una 01xx/02xx queda sin respuesta.

Selección y edición

Control	Uso
Tablas H2	Lista las tablas reutilizables con columna REQUEST/REQUEST_JSON; AUDIT_LOG se oculta.
Suite / dialect	Filtra por suite_id. Es una clasificación H2 y no necesariamente el dialecto wire.
Registros	Muestra hasta 200 filas de altura fija; al seleccionar una, carga REQUEST.
Reload	Descarta la edición del área JSON y vuelve a cargar el registro.
Update	Actualiza en H2 el registro seleccionado mediante un popup validado.
Save as	Inserta y confirma en H2 una nueva fila; el alta persiste al reiniciar.
Send	Parsea el JSON, aplica cripto si está habilitado, envía y muestra el outcome.
Reversar (0420)	Construye un reverso sobre la última transacción y lo envía.

4.1 Respuesta y validación de Adquirencia

El perfil de validación se construye desde `expected_rc` e `ignore_fields` del registro. También controla estructura, MTI de respuesta y eco de campos.

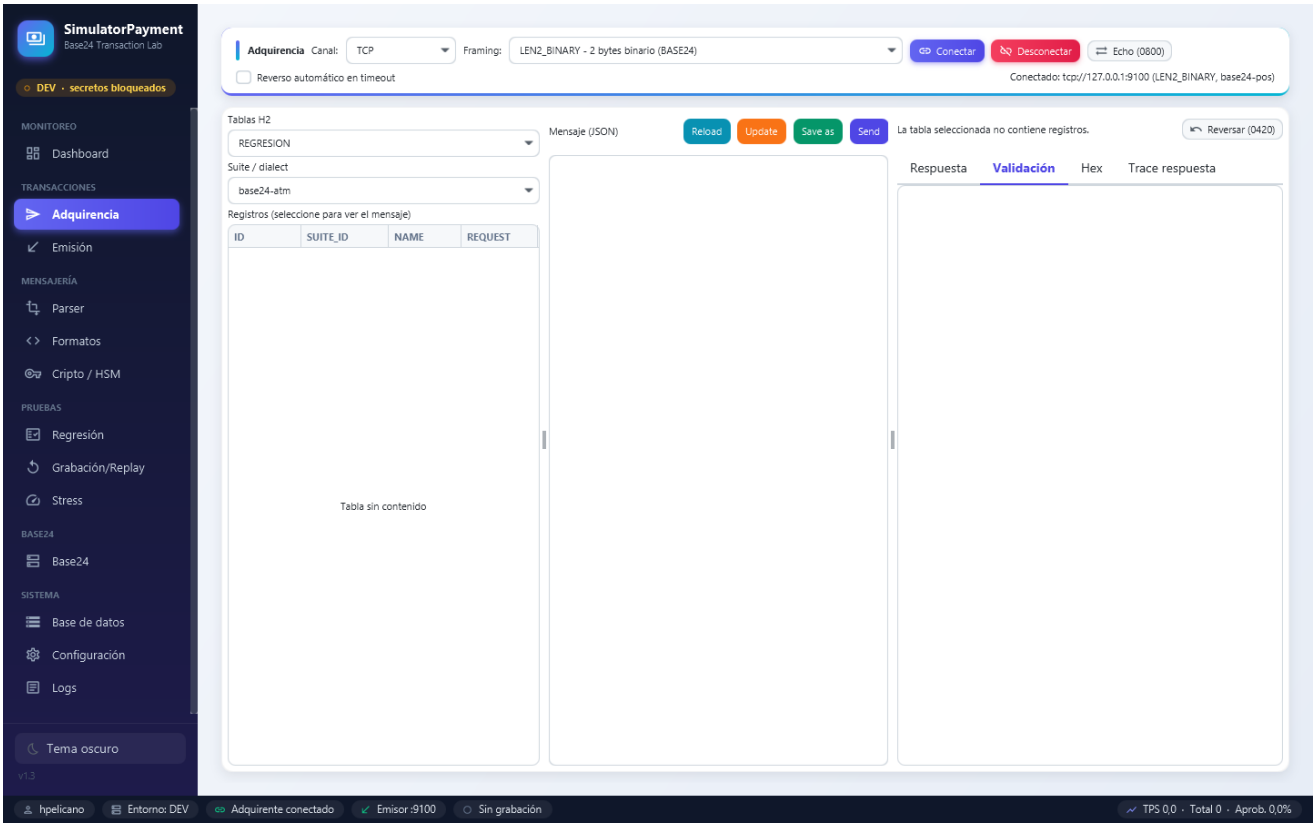


Figura 6. Pestaña Validación: resultado, controles configurados y observaciones.

Interpretación

- "VALIDACIÓN OK" significa que se cumplieron todos los controles activos; no implica que el RC sea 00 si se esperaba otro RC.
- Los campos de `ignore_fields` se excluyen del control de eco; se aceptan números separados por coma, punto y coma o espacio.
- Un timeout o error de transporte produce un outcome TIMEOUT/ERROR y no una respuesta ISO válida.
- El audit log recibe request, response, status y timestamp; el transaction log conserva datos enmascarados y latencia.

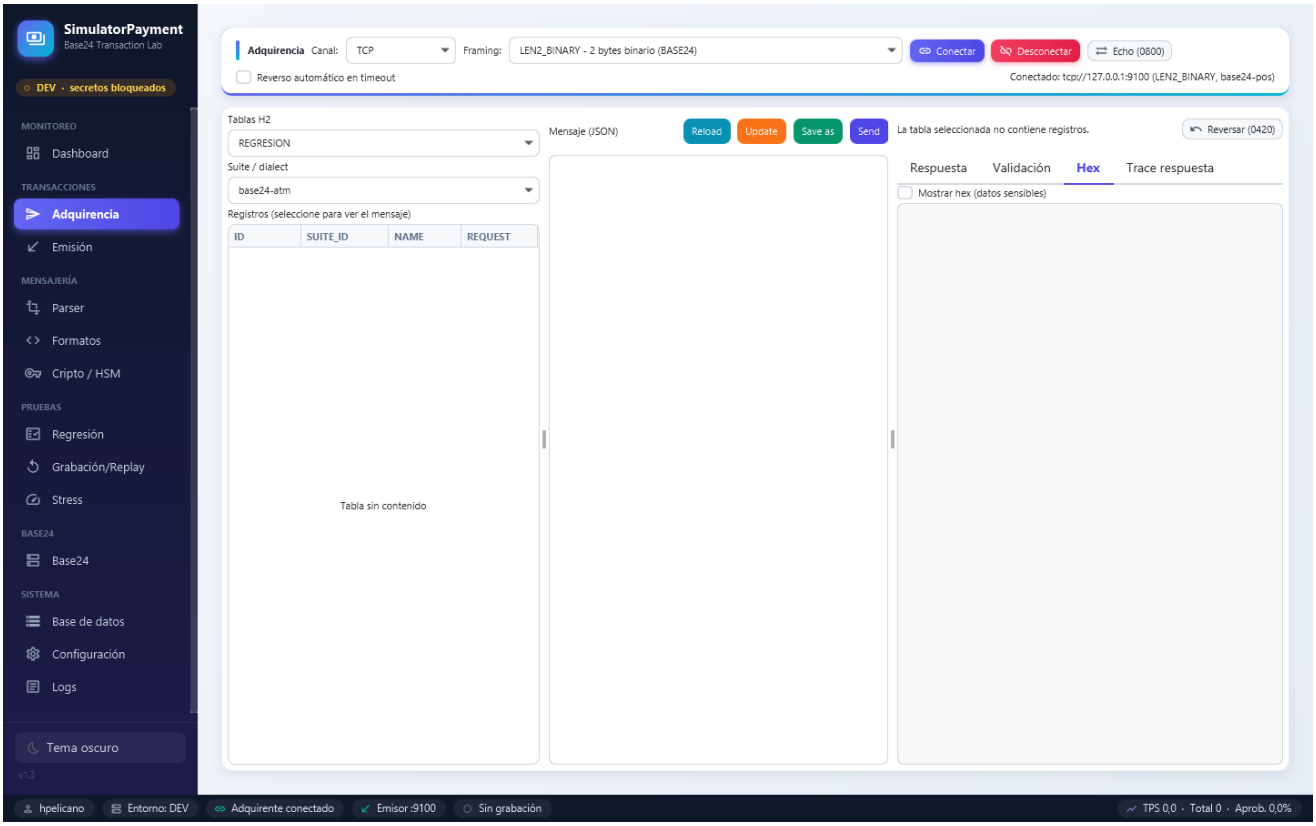


Figura 7. Pestaña Hex. El contenido crudo permanece oculto hasta habilitar “Mostrar hex”.

PCI. El wire crudo puede contener PAN, Track 2 y PIN block. No se persiste; sólo debe mostrarse en un ambiente controlado.

4.2 Trace de respuesta

Trace descompone el wire recibido en elemento, nombre, offset, longitud, valor y hex. Es la herramienta indicada cuando el JSON parece correcto pero hay diferencias de bitmap, longitud variable, encoding o header.

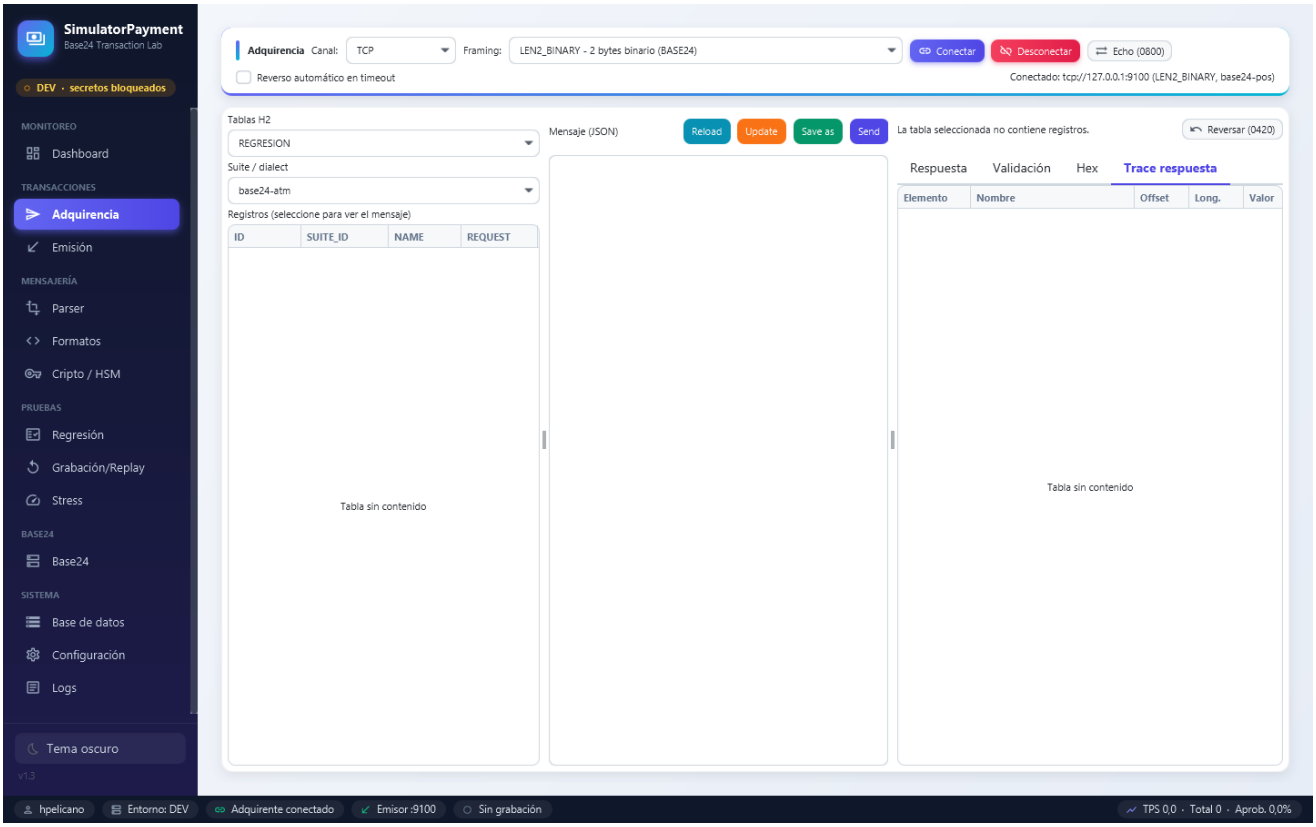


Figura 8. Trace de la respuesta, generado por el codec del dialecto.

Diagnóstico recomendado

- Confirme primero que el dialecto del endpoint coincide con el del host.
- Verifique el framing por separado: un framing erróneo suele impedir que el mensaje llegue al codec.
- Compare offsets y hex del primer campo divergente; una longitud LLVAR/LLLVAR incorrecta desplaza todos los campos posteriores.
- Revise encoding ASCII, BCD o EBCDIC, bitmap primario/secundario, header y sub-formatos compuestos.

5. Emisión — visión general

Emisión puede operar como Cliente o Server. Ambos modos tienen conexiones independientes de Adquirencia y un log exportable con dirección, canal, MTI, STAN, PC, RC, nota y JSON enmascarado.

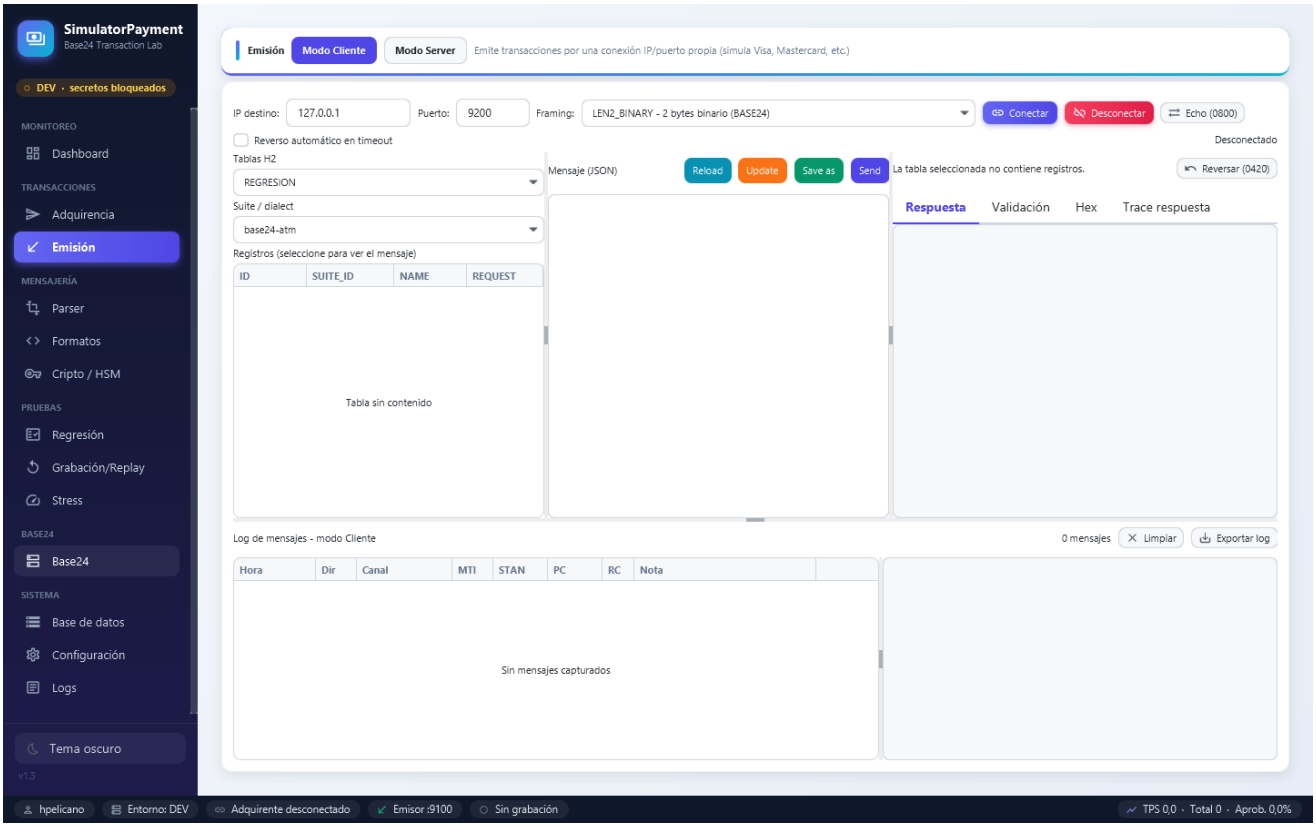


Figura 9. Modo Cliente: flujo H2 equivalente a Adquirencia, con IP/puerto/framing propios.

Modo Cliente

- Simula una marca o red (Visa, Mastercard u otra) que inicia transacciones hacia un host.
- Usa `issuerClient` como endpoint inicial, source estadístico `ISSUER_CLIENT` y su propio canal.
- Incluye Tablas H2, Suite/dialect, editor JSON, Reload, Update, Save as, Send, Echo, reverso y validación.
- Los outcomes también pueden alimentar grabación, estadísticas y transaction log.

5.1 Emisión Server y ruteo

Modo Server escucha requests en puerto 1, evalúa reglas ordenadas y genera una respuesta. El selector de ruteo determina si esa respuesta vuelve por el mismo socket o por un enlace separado.

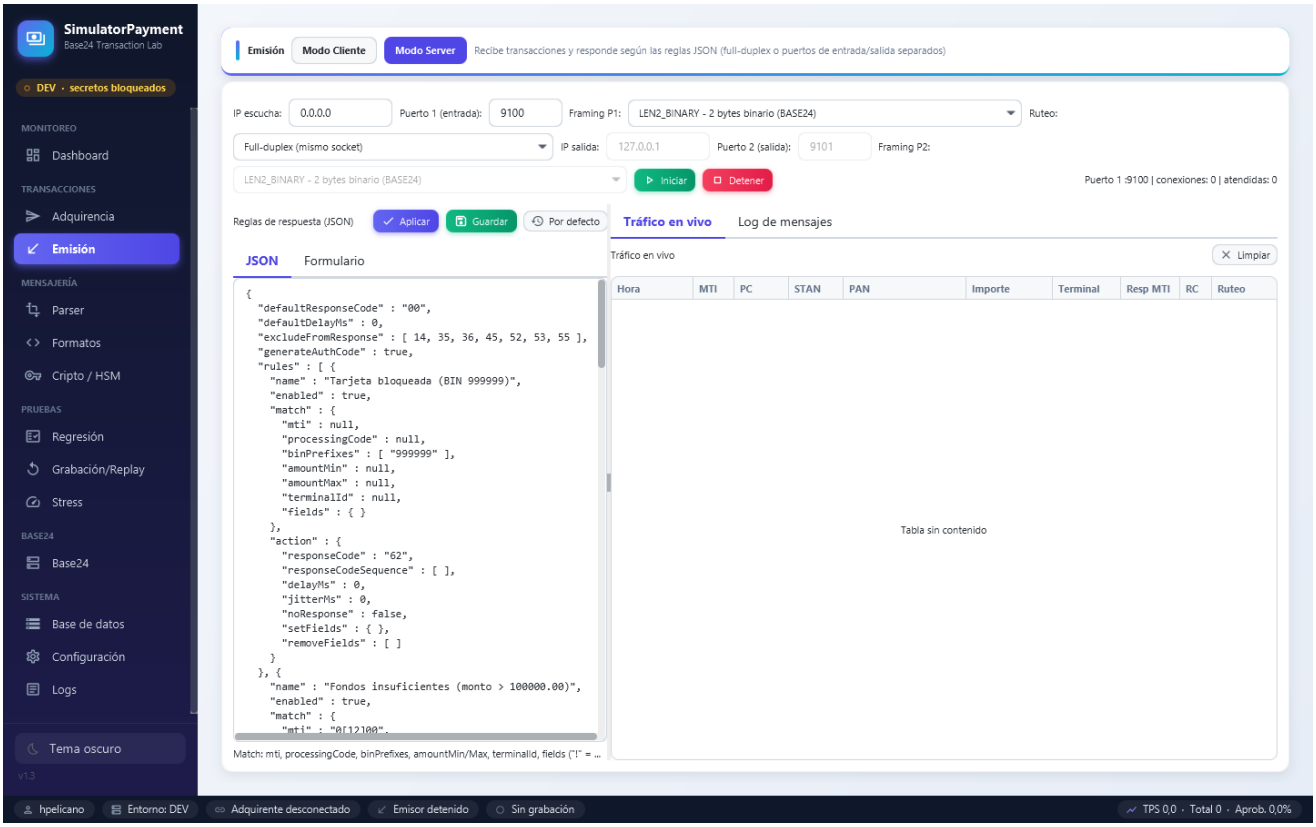


Figura 10. Modo Server: endpoint de entrada, framing P1, ruteo y endpoint/framing P2.

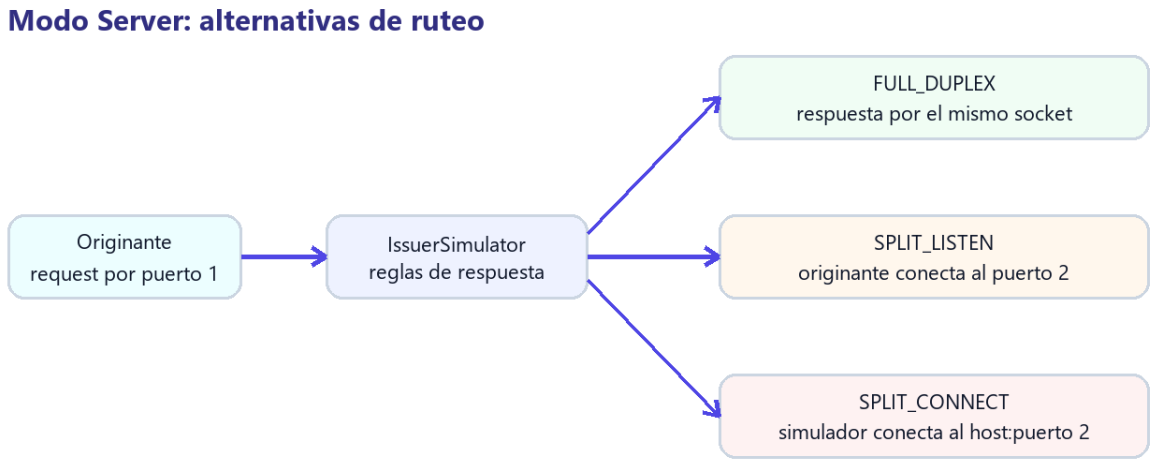


Figura 11. FULL_DUPLEX, SPLIT_LISTEN y SPLIT_CONNECT.

Ruteo	Comportamiento	Uso
FULL_DUPLEX	Response por la misma conexión en que ingresó el request.	Modelo TCP request/response habitual.
SPLIT_LISTEN	El simulador abre puerto 2 y hace broadcast a los peers conectados.	El originante mantiene un canal adicional de recepción.
SPLIT_CONNECT	El simulador conecta al host:puerto 2 y envía one-way.	El originante expone un listener de respuestas.

Ruteo separado. Si no existe peer en SPLIT_LISTEN o no se puede conectar en SPLIT_CONNECT, la respuesta no se entrega; se incrementa routeErrorCount y se registra un evento ERR en el log.

5.2 Reglas de respuesta del emisor

Las reglas se editan como JSON o mediante Formulario y se sincronizan al cambiar de pestaña. Se aplica la primera regla habilitada que coincida; si ninguna coincide, se usa defaultResponseCode/defaultDelayMs.

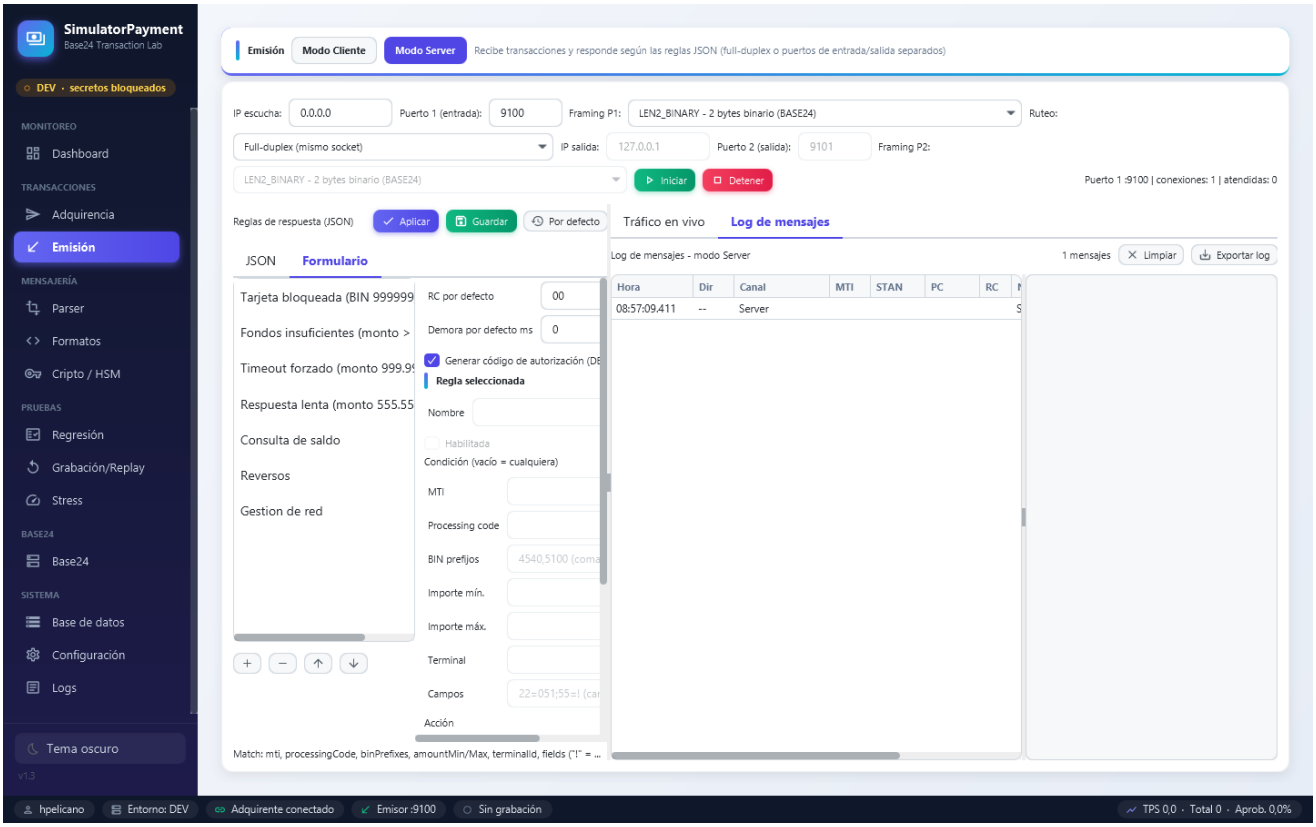


Figura 12. Editor visual de reglas.

Condiciones y acciones

Bloque	Atributos
Match	mti (admite regex), processingCode, binPrefixes, amountMin/amountMax en unidades mínimas, terminalId y mapa fields. El va
Action	responseCode, secuencia rotativa de RC, delayMs, jitterMs, noResponse, setFields con ${REQ:n}/${RANDOM_N:n} y removeFiel
Global	defaultResponseCode, defaultDelayMs, excludeFromResponse y generateAuthCode (DE38).

Reglas incluidas

- BIN 999999 → RC 62 (tarjeta bloqueada).
- Monto superior a 100000.00 → RC 51 (fondos insuficientes).
- Monto 999.99 → noResponse para provocar timeout.
- Monto 555.55 → demora de 3000 ms.
- Incluye comportamientos para consulta de saldo, reversos y gestión de red.

Persistencia

- “Aplicar” reemplaza las reglas activas en memoria.
- “Guardar” escribe `config/issuer-rules.json`.
- “Por defecto” restaura la definición embebida; guarde si desea que sobreviva al reinicio.

5.3 Tráfico y log de Emisión

"Tráfico en vivo" resume requests y respuestas. "Log de mensajes" conserva bloques detallados que se pueden exportar a un archivo .log para documentar la sesión.

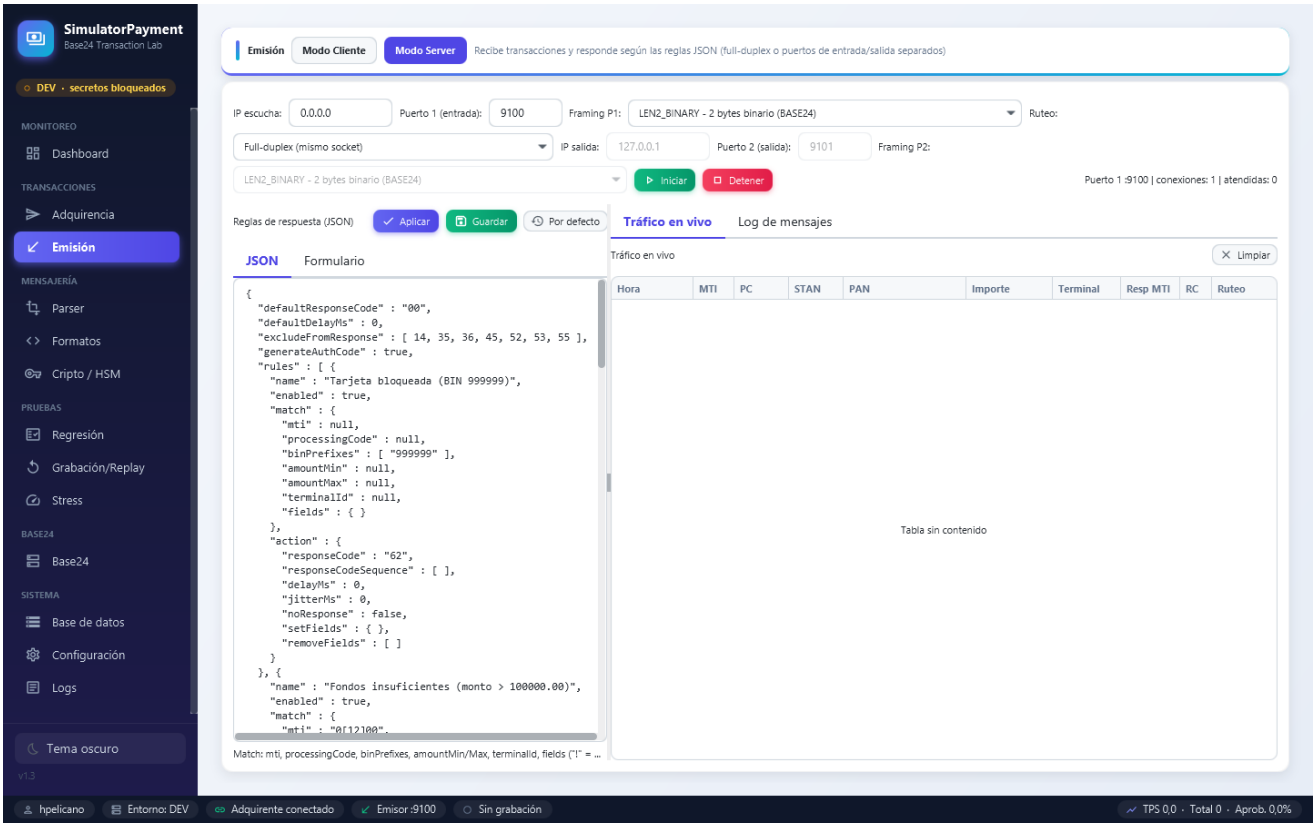


Figura 13. Tráfico en vivo: hora, MTI, PC, STAN, PAN enmascarado, importe, terminal, RC, ruteo, regla y demora.

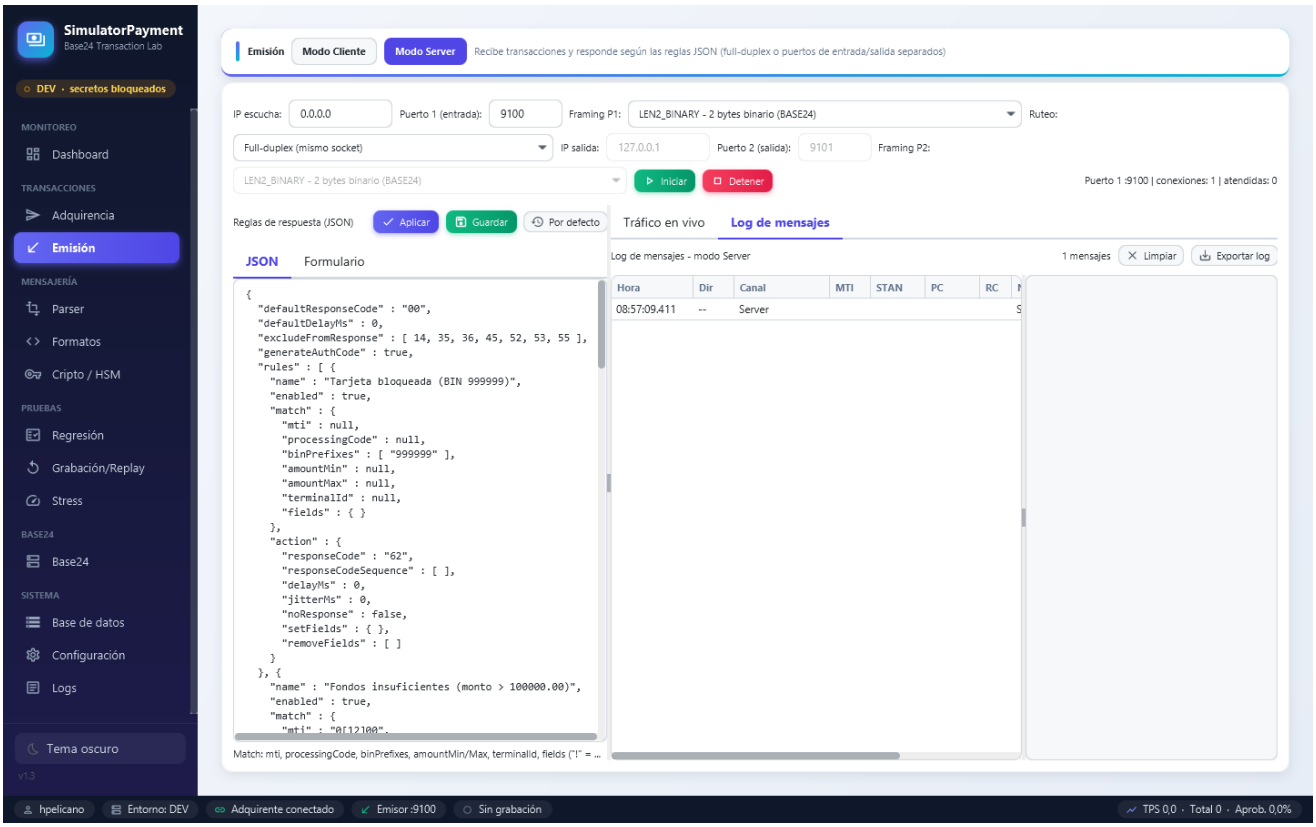


Figura 14. Log de mensajería: request, response, timeout y errores de ruteo.

Seguridad del log

- PAN, tracks y PIN se enmascaran antes de incorporar el JSON al log.
- El buffer admite hasta 5000 entradas y descarta las más antiguas.
- La exportación genera `emision-cliente-YYYYMMDD-HHmmss.log` o `emision-server-…`.
- El log de pantalla es específico de cada modo; el log general de la aplicación se consulta en la vista Logs.

6. Parser — JSON, wire y diagnóstico

Parser transforma JSON a hex/wire, decodifica hex a JSON, construye trace, valida estructura y convierte entre dialectos. Es independiente de una conexión de red.

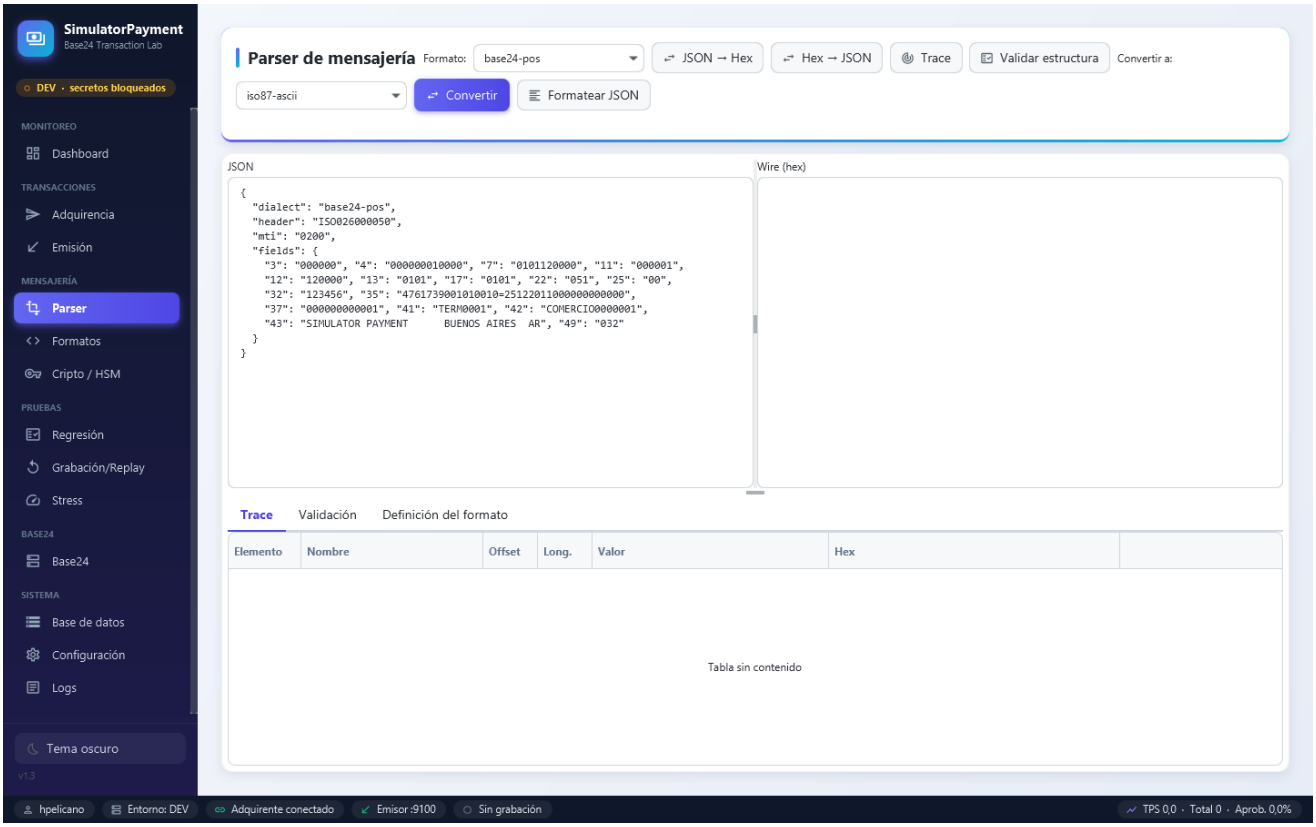


Figura 15. Parser con selector de dialecto, JSON, hex y herramientas.

Acciones

Acción	Descripción
JSON → Hex	fromJson + pack; usa el dialect del JSON o el seleccionado, y muestra los bytes del payload ISO.
Hex → JSON	Decodifica con el dialecto seleccionado. El framing de red no forma parte de este hex.
Trace	Desglosa header, MTI, bitmap y campos con offset/longitud/hex.
Validar estructura	Ejecuta validación del mensaje contra la definición resuelta del formato.
Convertir	Copia campos compatibles hacia el dialecto destino; no garantiza equivalencia semántica total.
Formatear JSON	Indenta y normaliza el contenido para lectura.

Framing. El Parser opera sobre el payload ISO8583. Si capturó un stream TCP, retire primero el prefijo LEN2/LEN4 y los bytes reservados que correspondan.

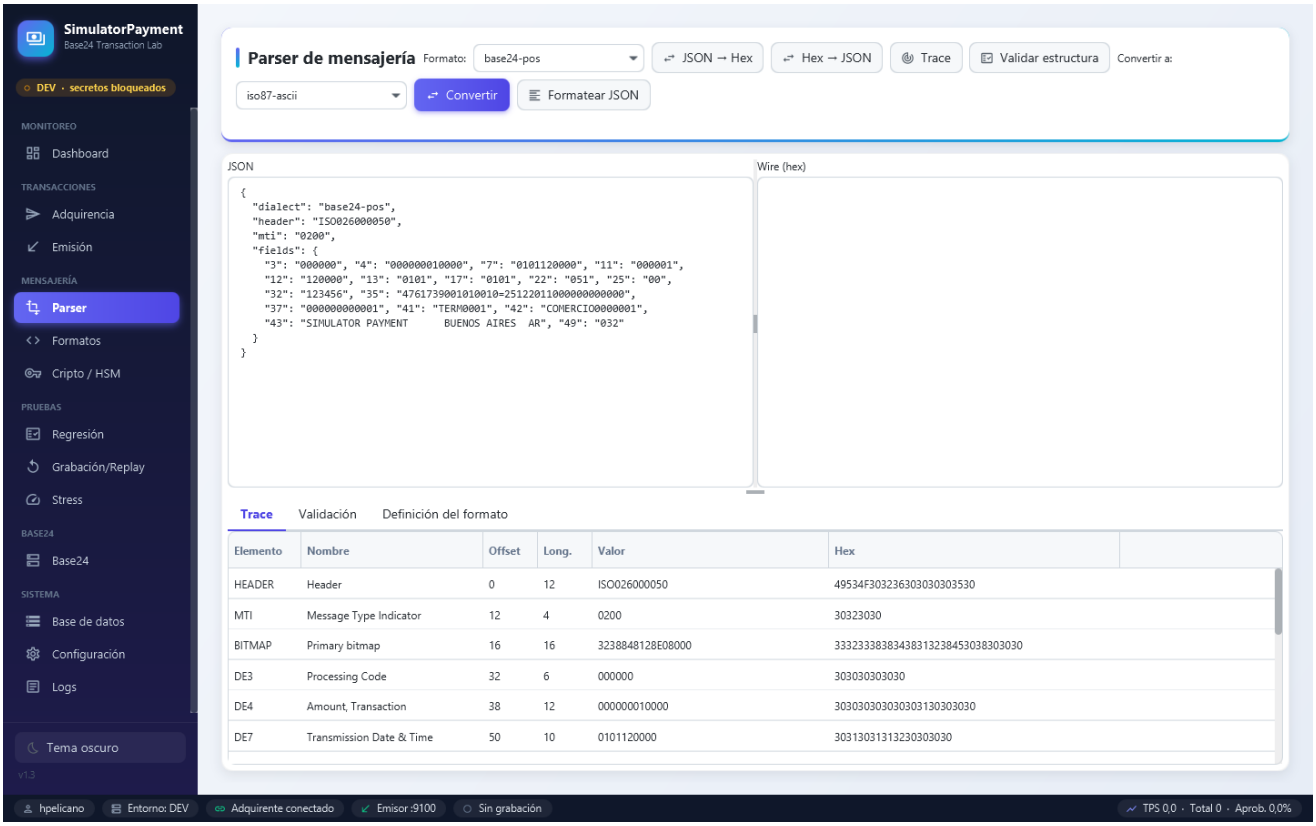


Figura 16. Trace y validación estructural del payload.

7. Formatos — definición de dialectos

Formatos administra las reglas que convierten IsoMessage/JSON en bytes. Cada formato posee un id de dialecto, propiedades globales y definiciones por campo. Guardar produce un dialecto autónomo en `config/formats/<id>.json`.

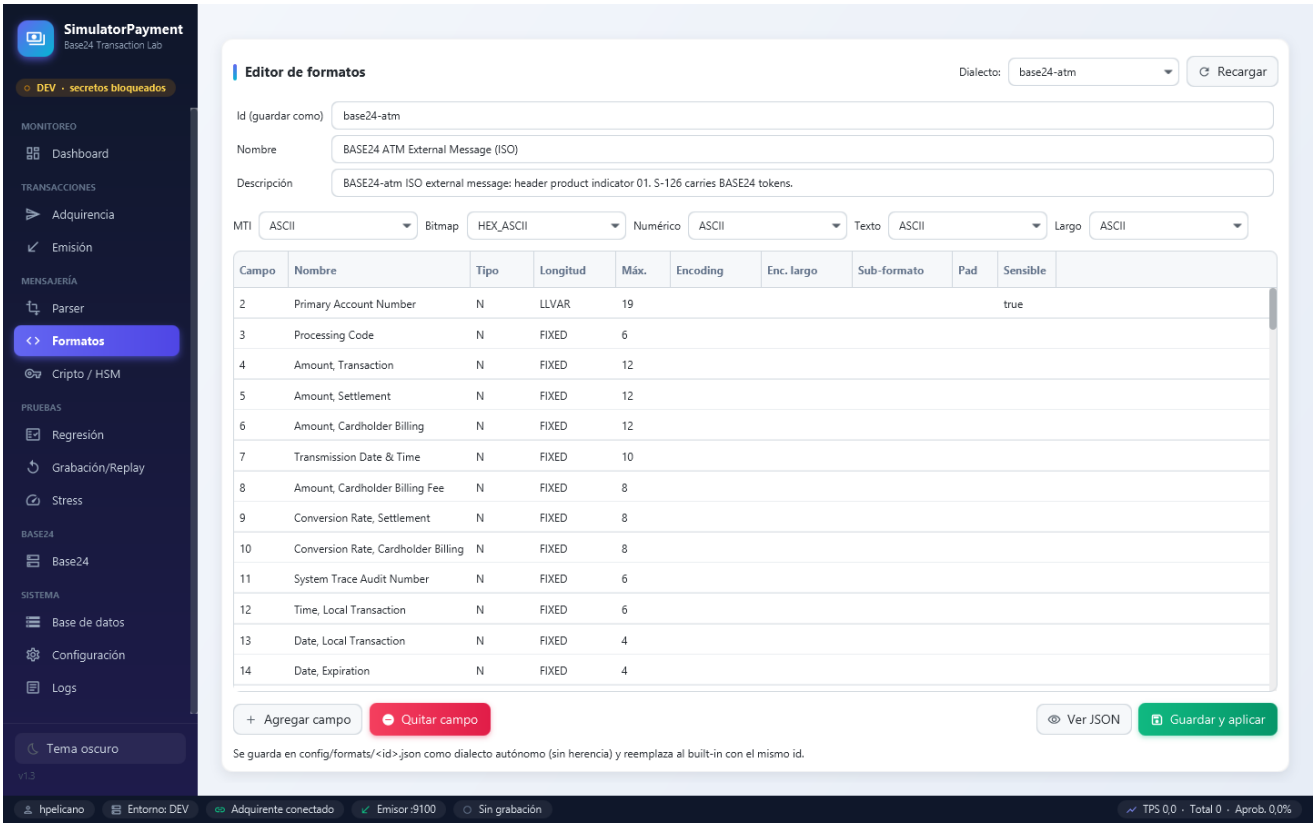


Figura 17. Editor de formatos/dialectos.

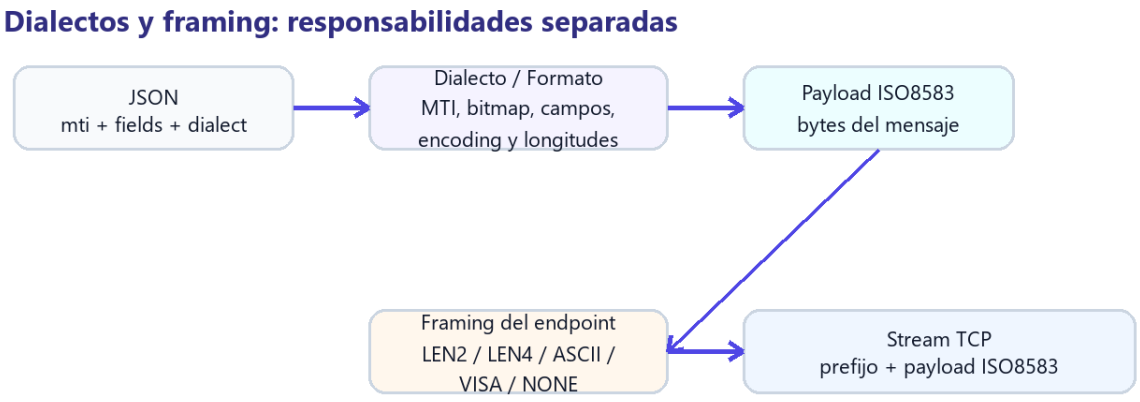


Figura 18. El dialecto codifica el payload; el framing delimita el payload en TCP.

Qué configura un dialecto

- MTI: longitud, encoding y reglas de request/response.
- Bitmap primario/secundario y orden de campos.
- Tipo de campo, longitud fija o variable, prefijo LLVAR/LLLVAR, encoding ASCII/BCD/EBCDIC y padding.

- Header, campos sensibles, sub-formatos y herencia resuelta.
- Nombre e id. Un archivo de usuario con el mismo id reemplaza al built-in.

Flujo al guardar

- La vista valida id y definición, escribe JSON en el directorio de configuración, registra el dialecto y resuelve herencias.
- Se limpia el cache de codecs para que el cambio se aplique en caliente.
- Cambiar longitudes/encoding puede romper mensajes existentes; pruebe JSON → Hex, Hex → JSON y Trace antes de conectar.
- El atributo framing preservado dentro del dialecto no controla el socket. El transporte usa el framing del endpoint.

7.1 Catálogo de dialectos incluidos

El registro carga formatos embebidos y luego los archivos del usuario. Los ids exactos disponibles dependen de la versión; entre los incorporados se encuentran los siguientes perfiles.

Dialect id	Familia / uso
iso87-ascii	ISO 8583:1987 con representación ASCII.
iso87-binary	ISO 8583:1987 con bitmap/campos binarios según definición.
iso93-ascii	ISO 8583:1993 ASCII.
base24-pos	Mensajería Base24 para POS; formato de laboratorio principal.
base24-atm	Mensajería Base24 para ATM.
base24-upay / base24-clover / base24-chu	Variantes de integración agrupadas por suite/dialect.
visa-base1	VISA Base I; suele combinarse con LEN2_BINARY_HEADER.
mastercard-cis	Mastercard Customer Interface Specification.
hpdh / bnet	Plantillas de HPDH/BNET que deben ajustarse a la especificación del sitio.

Suite ≠ dialecto. suite_id en H2 agrupa casos. Puede llamarse base24-regresion o base24-stress sin ser un codec real. La codificación se decide por el campo dialect del mensaje o el endpoint.

Estrategia para crear un formato

- Copie el formato más cercano y asigne un id nuevo.
- Implemente primero header, MTI y bitmap; luego campos mínimos de un 0200/0210.
- Valide round-trip JSON → wire → JSON y trace con vectores oficiales.
- Marque PAN, tracks, PIN y material criptográfico como sensibles.
- Documente framing, dialecto y correlación en la configuración del ambiente.

8. Framing TCP

El framing es el prefijo que permite separar mensajes dentro del stream TCP. Debe coincidir exactamente en ambos extremos. Se selecciona en Adquirencia TCP, Emisión Cliente y Emisión Server (P1/P2), y toma como valor inicial el endpoint del ambiente.

Valor	Prefijo para 300 bytes	Semántica / límite
LEN2_BINARY	01 2C	2 bytes big-endian; excluye header. BASE24 default. Hasta 65535.
LEN2_BINARY_INCLUSIVE	01 2E	2 bytes big-endian; incluye sus 2 bytes.
LEN2_BINARY_HEADER	01 2C 00 00	VISA: 2 bytes de largo + 2 reservados; largo excluye los 4.
LEN4_BINARY	00 00 01 2C	4 bytes big-endian; excluye header.
LEN4_ASCII	"0300"	4 dígitos ASCII; excluye header; payload máx. 9999.
LEN4_ASCII_INCLUSIVE	"0304"	4 dígitos ASCII; incluye header; payload máx. 9995.
NONE	sin prefijo	Cada lectura TCP se toma como un mensaje; no es robusto con fragmentación/pipelining

Reglas prácticas

- TCP no conserva fronteras de mensaje; use NONE sólo cuando el protocolo superior garantice una lectura completa.
- Un prefijo inclusivo agrega al valor el tamaño de su propio header, no cambia el payload ISO.
- P1 y P2 pueden tener framing diferente en Emisión Server, aunque normalmente se recomienda homogeneidad.
- El selector de UI aplica a la conexión actual y no reescribe environments.json.
- ConfigService valida que el string de framing sea uno de los valores admitidos.

9. Cripto / HSM — configuración

El HSM de software permite ensayar PIN blocks, MAC, CVV y criptogramas EMV. Se integra como interceptor de requests en Adquirencia/Emisión Cliente y como postprocesador de responses en Emisión Server.

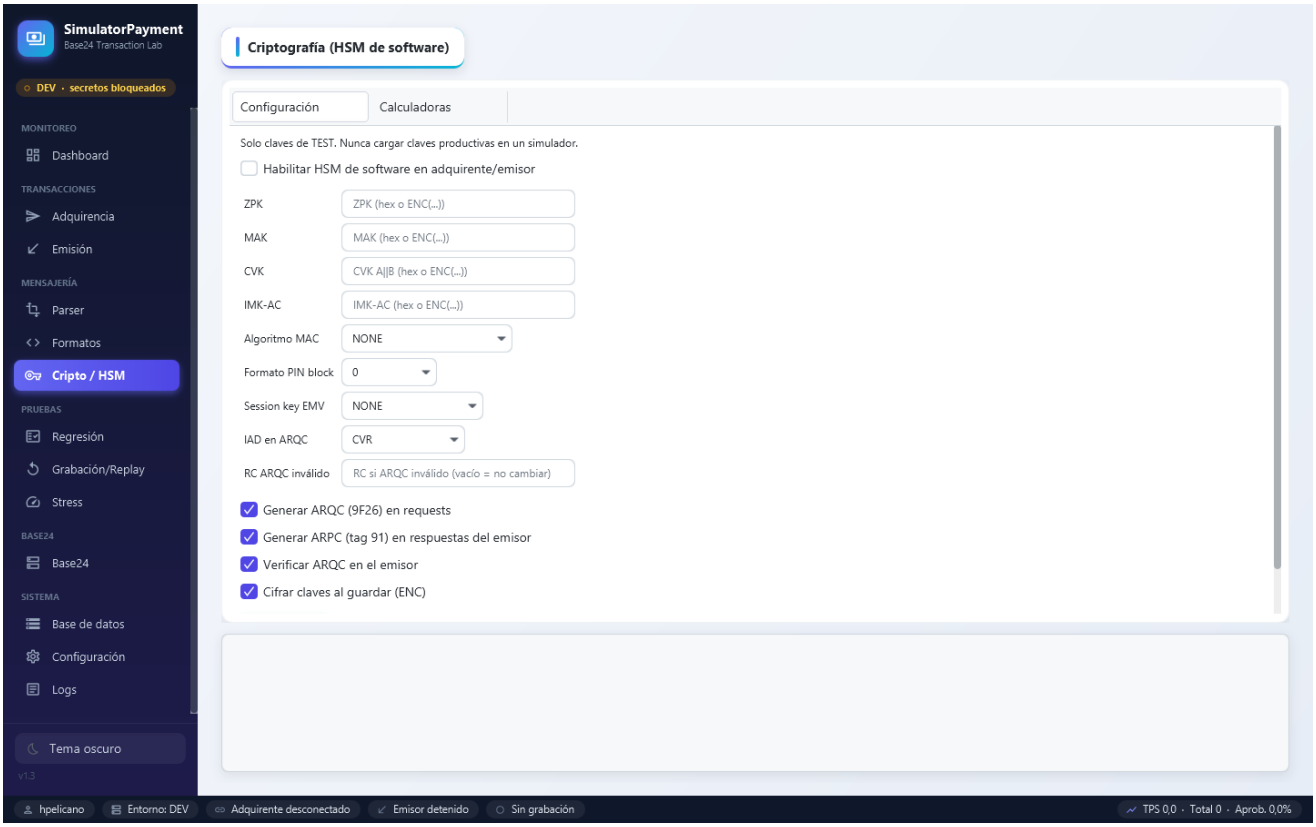


Figura 19. Configuración del HSM de software.

Claves y opciones

Elemento	Uso
ZPK	Cifra/descifra PIN blocks ISO formatos 0, 1 y 3.
MAK	Genera/verifica MAC X9.19, ISO9797 ALG1 o AES-CMAC.
CVK	Calcula CVV, CVV2 e iCVV.
IMK-AC	Deriva session keys EMV y calcula/verifica ARQC/ARPC.
KCV	Valor de verificación para comprobar que la clave ingresada es la esperada.
ENC(...)	Las claves se pueden guardar cifradas con la contraseña maestra.

- DE52 con valor simbólico PIN:1234 puede reemplazarse por un PIN block cifrado.
- DE55 con tags 9F26/9F36 permite generar o verificar ARQC.
- El emisor puede devolver un RC configurable ante ARQC inválido y generar tag 91 (ARPC).
- MAC puede ubicarse en DE64 o DE128 según formato/configuración.

9.1 Calculadoras criptográficas

Las calculadoras permiten validar vectores de forma aislada antes de activar el pipeline automático.

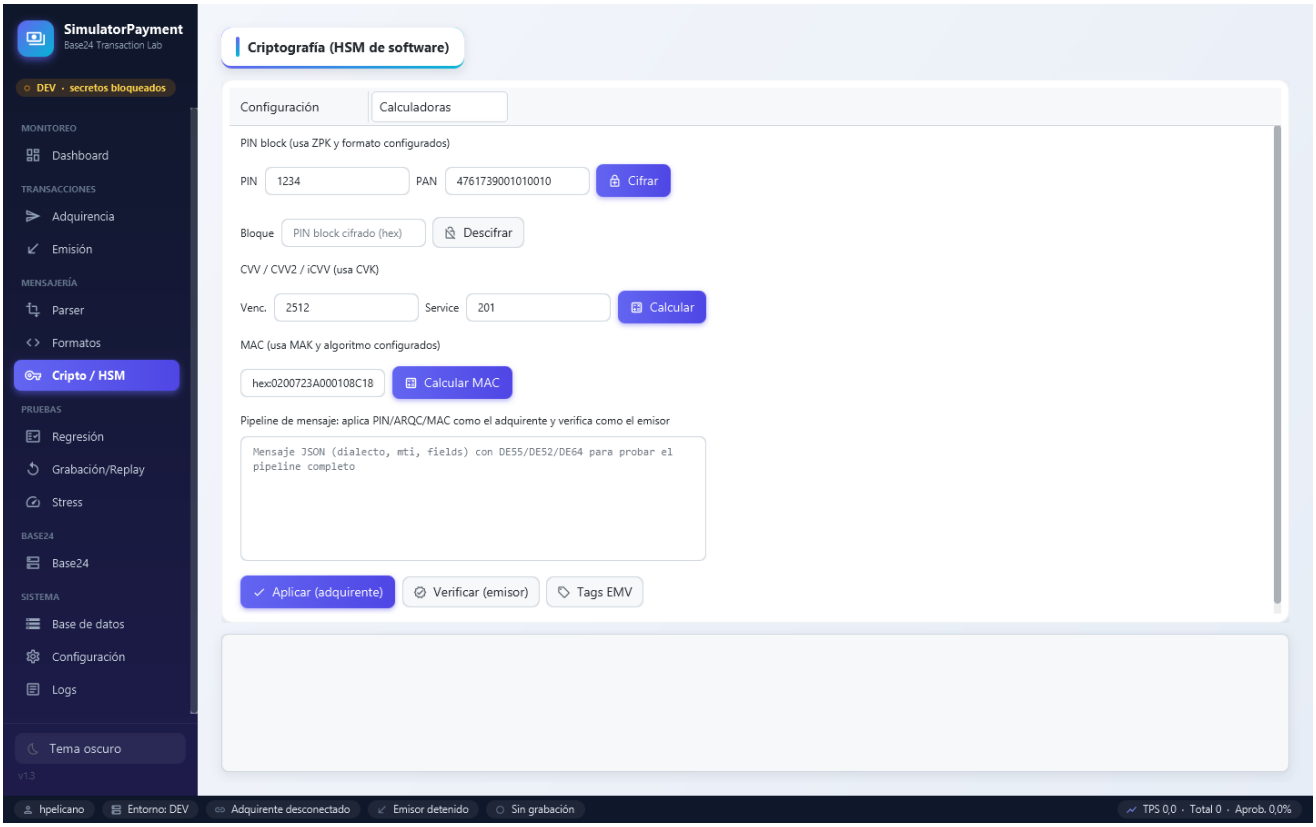


Figura 20. Herramientas de PIN, CVV, MAC, ARQC/ARPC y KCV.

Buenas prácticas

- Use exclusivamente claves de laboratorio; no copie claves de producción.
- Compare KCV al importar una clave y documente el algoritmo exacto.
- Para EMV, controle PAN/PSN, ATC, UN, monto, moneda, país, tipo de transacción, TVR y AIP.
- Nunca exporte logs con PIN en claro ni DE52 sin enmascarar.
- El HSM de software sirve para simulación, no sustituye un HSM certificado.

10. Regresión — suites y casos

Regresión organiza casos persistentes por suite, ejecuta requests contra el canal de Adquirencia y genera resultados repetibles. Puede refrescar campos dinámicos y detenerse en el primer fallo.

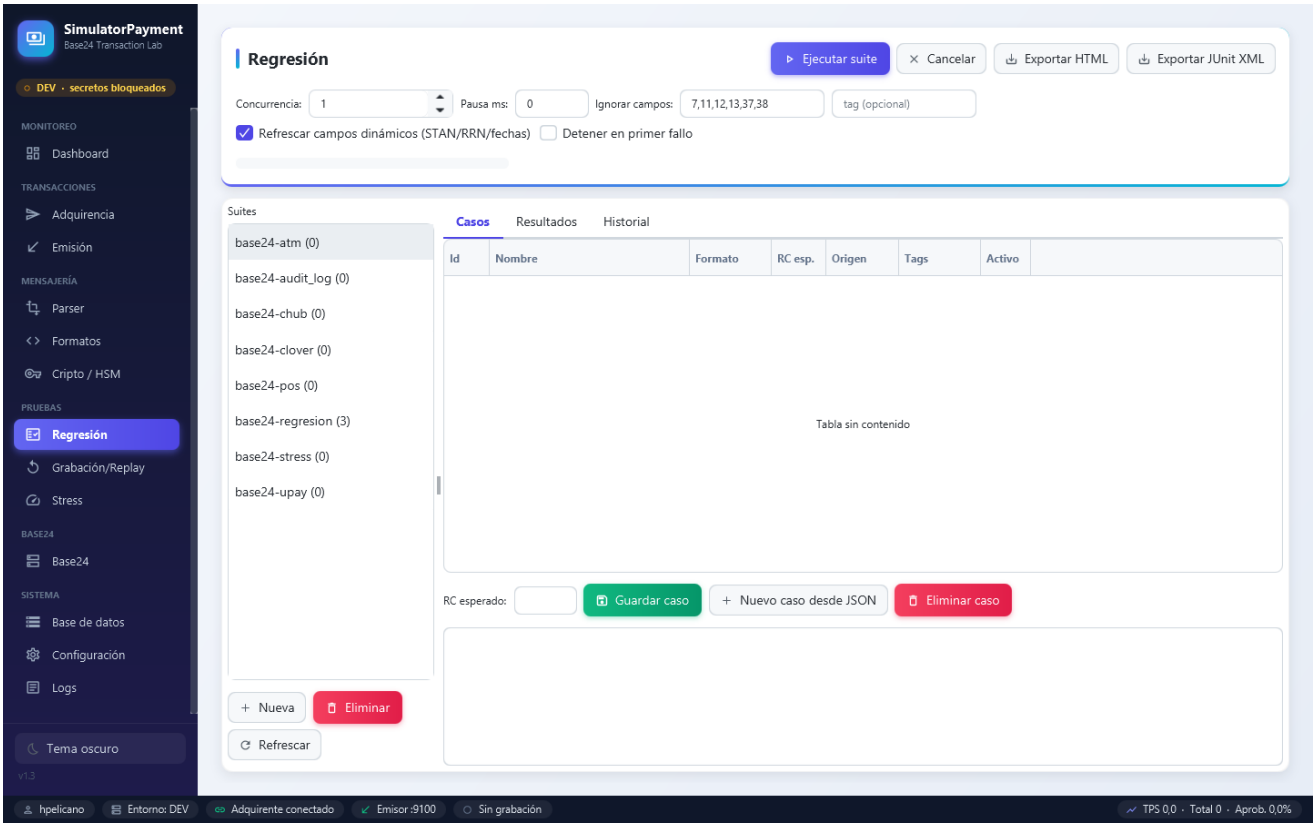


Figura 21. Gestión de suites y casos.

Preparación

- Cree o seleccione una suite y asocie casos con request JSON, expected_rc, ignore_fields y enabled.
- "Refrescar campos dinámicos" regenera STAN, RRN y fechas para evitar duplicados.
- "Detener en primer fallo" reduce ruido cuando una precondición general no se cumple.
- El canal de Adquirencia debe estar conectado antes de ejecutar.

Acciones

Control	Función
Nueva / Eliminar / Refrescar	Administra suites y recarga el catálogo.
Guardar caso	Persiste cambios del caso actual.
Nuevo caso desde JSON	Crea un caso a partir del editor.
Eliminar caso	Borra el caso seleccionado.
Ejecutar suite / Cancelar	Inicia o interrumpe la corrida.
Exportar HTML / JUnit XML	Genera evidencia navegable o integración CI.

10.1 Resultados e historial de regresión

Cada resultado conserva status, RC esperado/real, latencia, request/response y detalle de validación. Las corridas pueden compararse para distinguir regresiones de cambios esperados.

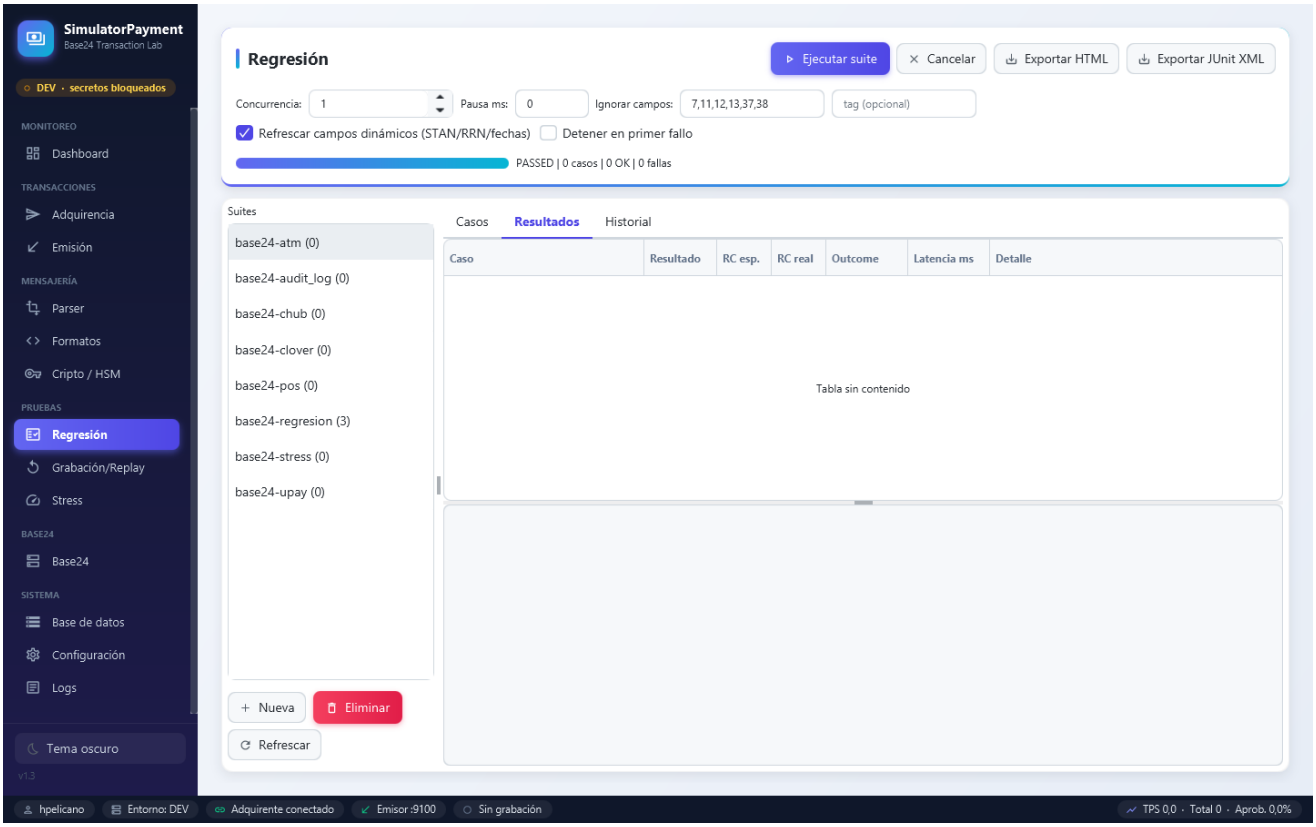


Figura 22. Resultados de la corrida.

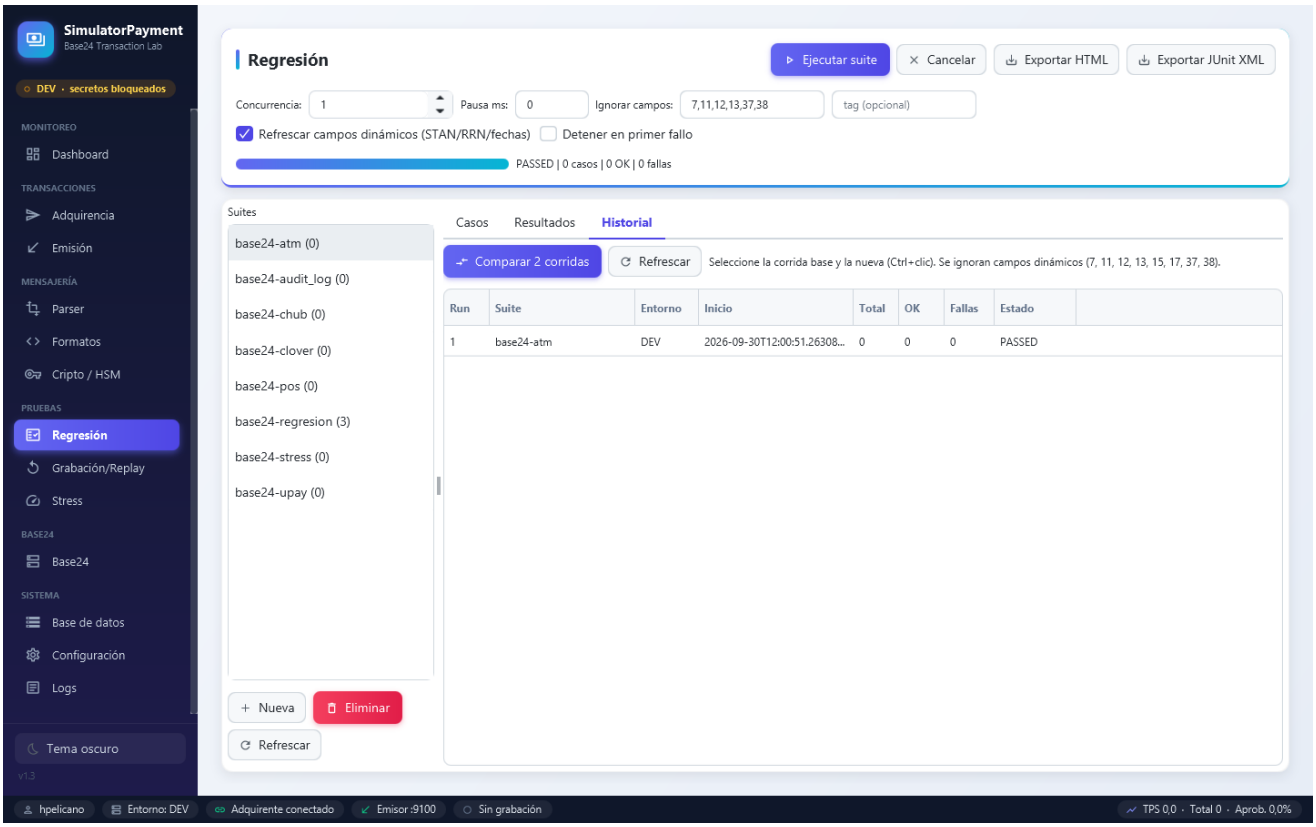


Figura 23. Historial y comparación de dos corridas.

Clasificación de comparación

- SAME: mismo comportamiento.
- REGRESSION / FIXED: empeoró o se corrigió respecto de la base.
- RC_CHANGED / FIELDS_CHANGED: cambió código o contenido.
- ONLY_IN_BASE / ONLY_IN_OTHER: el caso sólo existe en una corrida.
- Por defecto se ignoran campos dinámicos 7, 11, 12, 13, 15, 17, 37 y 38.

11. Grabación / Replay

Grabación/Replay captura tráfico real del adquirente y emisor y lo reproduce preservando o acelerando los tiempos. Las grabaciones se cifran con la contraseña maestra porque pueden contener mensajes en claro.

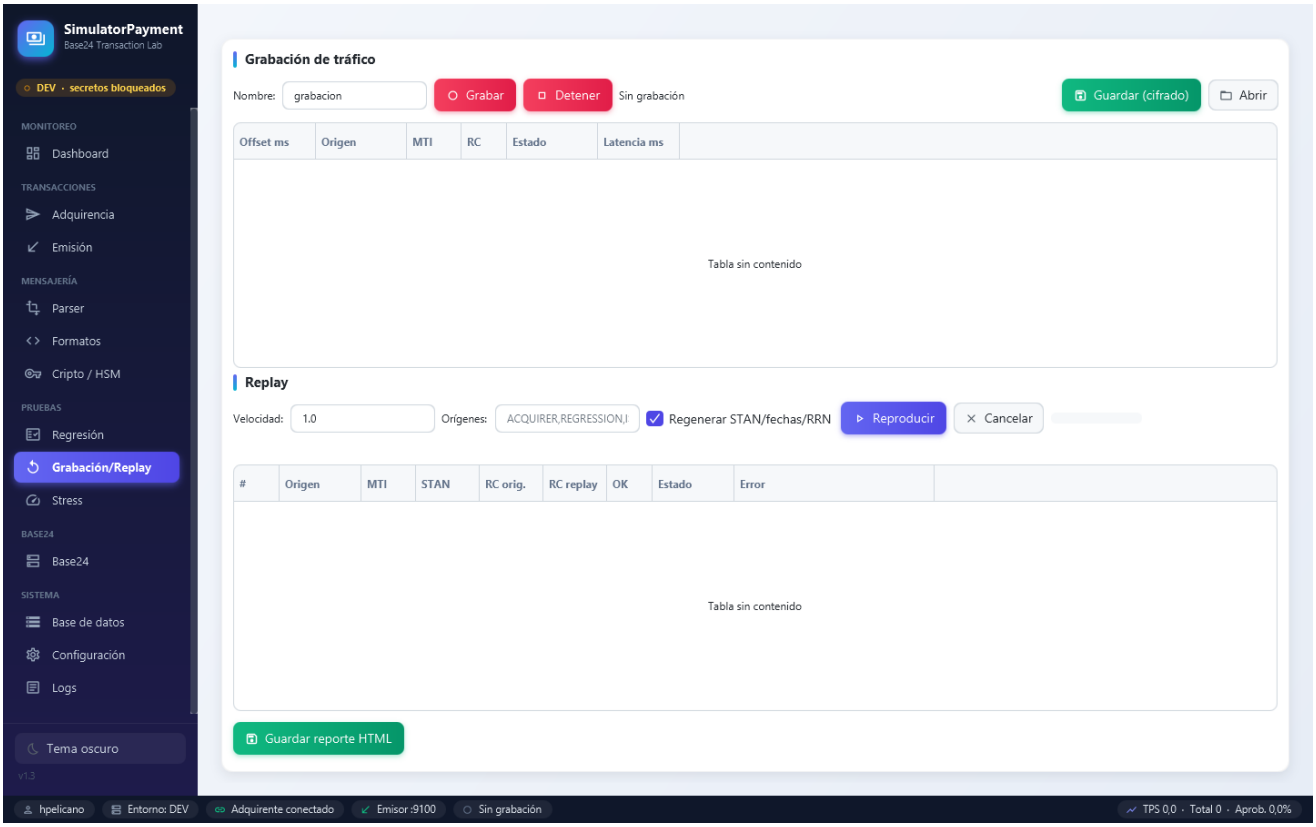


Figura 24. Grabación, apertura, filtros y replay.

Flujo de trabajo

- Presione “Grabar” antes del escenario y “Detener” al terminar.
- Guarde la captura cifrada. Si los secretos están bloqueados, se solicita la contraseña maestra.
- Abra una grabación, filtre por origen y configure velocidad (0 = máxima).
- Opcionalmente regenere STAN, fechas y RRN para evitar correlaciones duplicadas.
- Reproduzca contra el canal actual y compare RC original con RC de replay.
- Exporte el reporte HTML como evidencia.

Contenido sensible. Una grabación puede contener el mensaje original antes del enmascarado. Mantenga el archivo cifrado, controle su distribución y elimínelo cuando finalice la investigación.

12. Stress — carga y SLA

Stress genera carga de modelo abierto con perfiles CONSTANT, RAMP, STEP o SPIKE. Controla TPS objetivo, duración, máximo en vuelo y criterios de aborto por SLA.

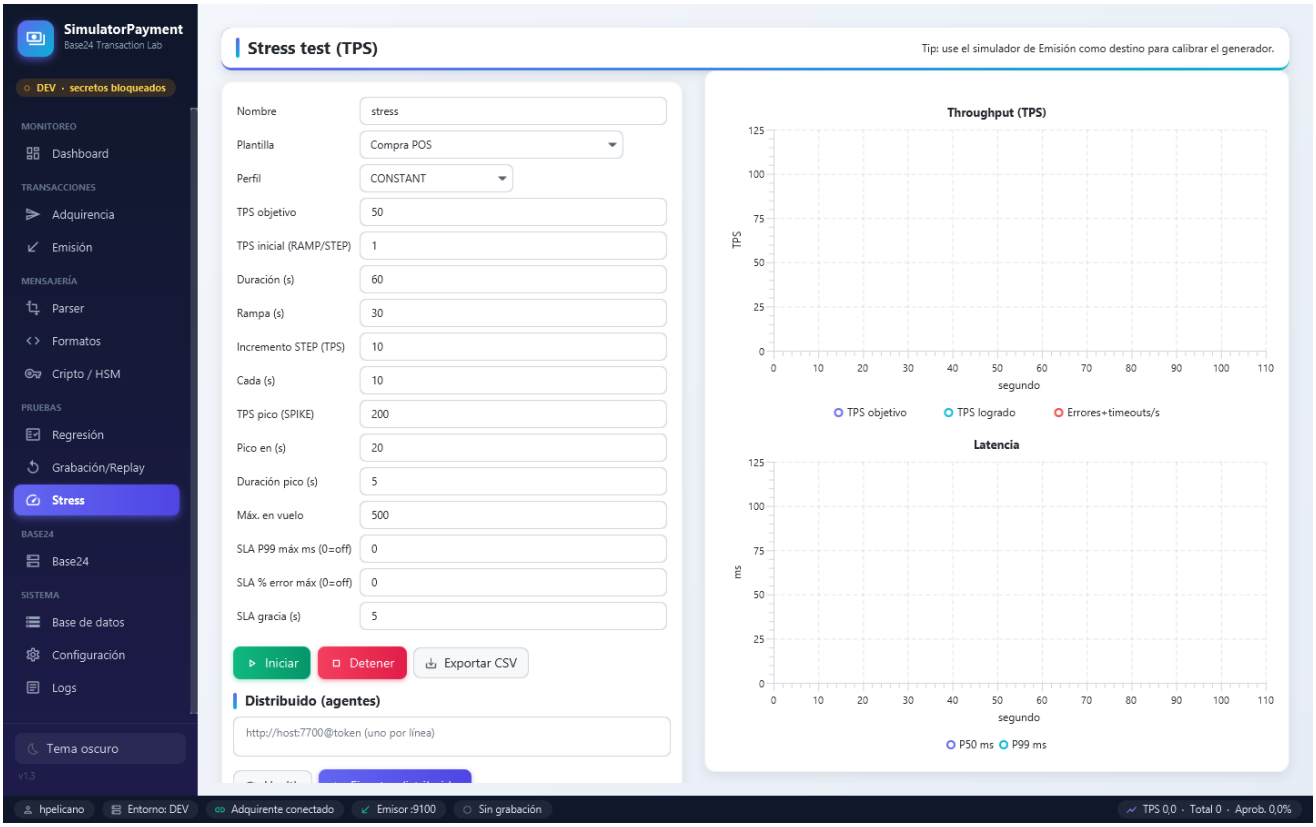


Figura 25. Configuración de una prueba de carga.

Parámetros principales

Parámetro	Descripción
Perfil	CONSTANT mantiene TPS; RAMP interpola; STEP incrementa por escalones; SPIKE aplica picos.
TPS / duración	Objetivo de generación y ventana de ejecución.
Máximo en vuelo	Backpressure para limitar requests pendientes.
SLA	Umbrales de error, timeout, latencia/percentiles o aprobación que pueden abortar.
Mensaje base	Template/caso a clonar, con actualización de campos dinámicos.

Lectura de resultados

- Sent, approved, declined, timeout y error deben analizarse junto con TPS efectivo.
- p95/p99 revelan colas y saturación que el promedio oculta.
- Una prueba válida debe incluir warm-up, ventana estable y cool-down.
- Exporte CSV para análisis y adjunte configuración exacta del ambiente.

12.1 Stress en ejecución y distribuido

Durante la ejecución se actualizan muestras y gráficos. El botón Detener cancela en forma controlada.

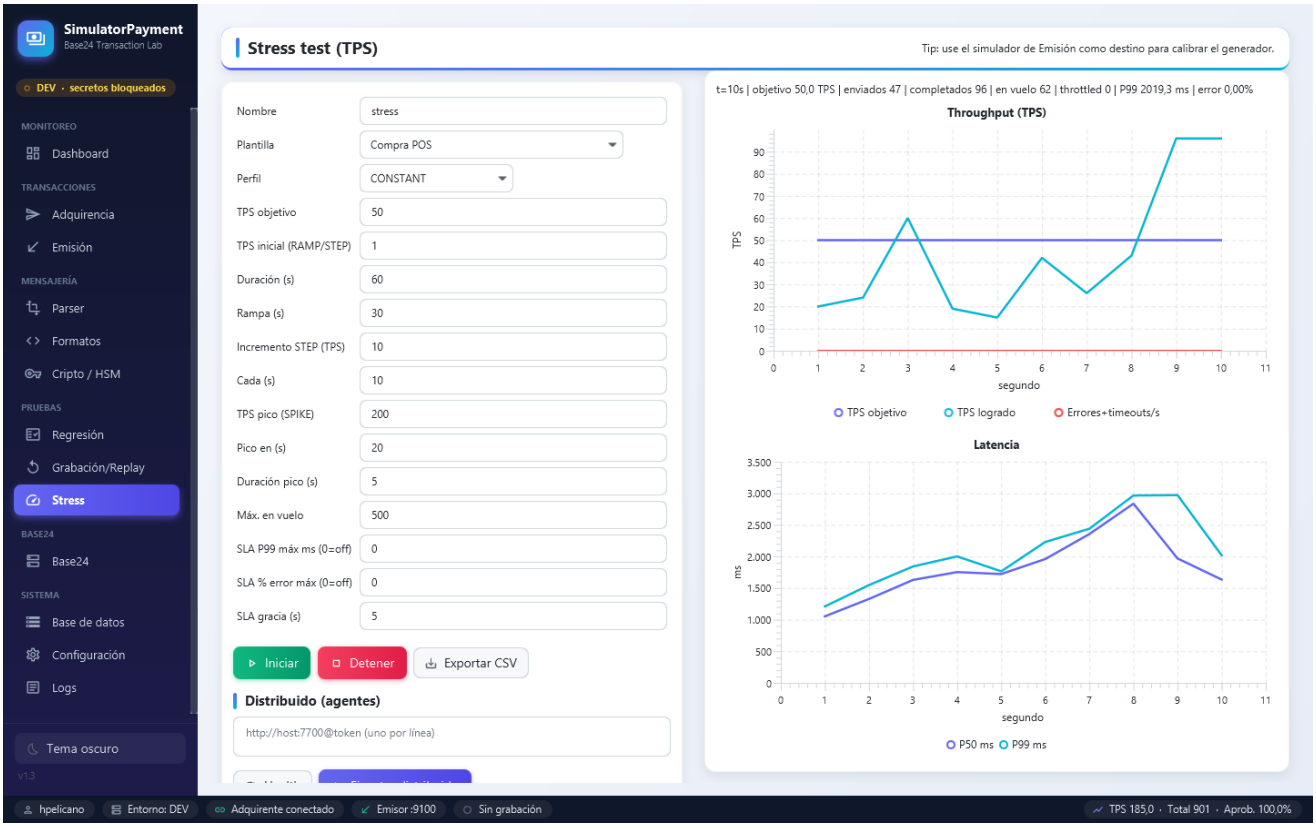


Figura 26. Prueba de stress en ejecución.

Agentes distribuidos

```
run.bat --headless agent --token S3cr3t0Agente --port 7700 --env QA
```

- Cargue una línea por agente en la pestaña Distribuido y utilice “Health”.
- “Ejecutar distribuido” divide el TPS en partes iguales y sincroniza el inicio.
- Los endpoints /health, /run, /status y /stop usan X-Agent-Token.
- Los conteos se suman y los percentiles agregados usan el máximo entre agentes como cota conservadora.

13. Base24 — Guardian y Pathway

La vista Base24 agrupa consultas Guardian, ABM y comandos Pathway. Las conexiones se crean de forma perezosa desde el ambiente activo.

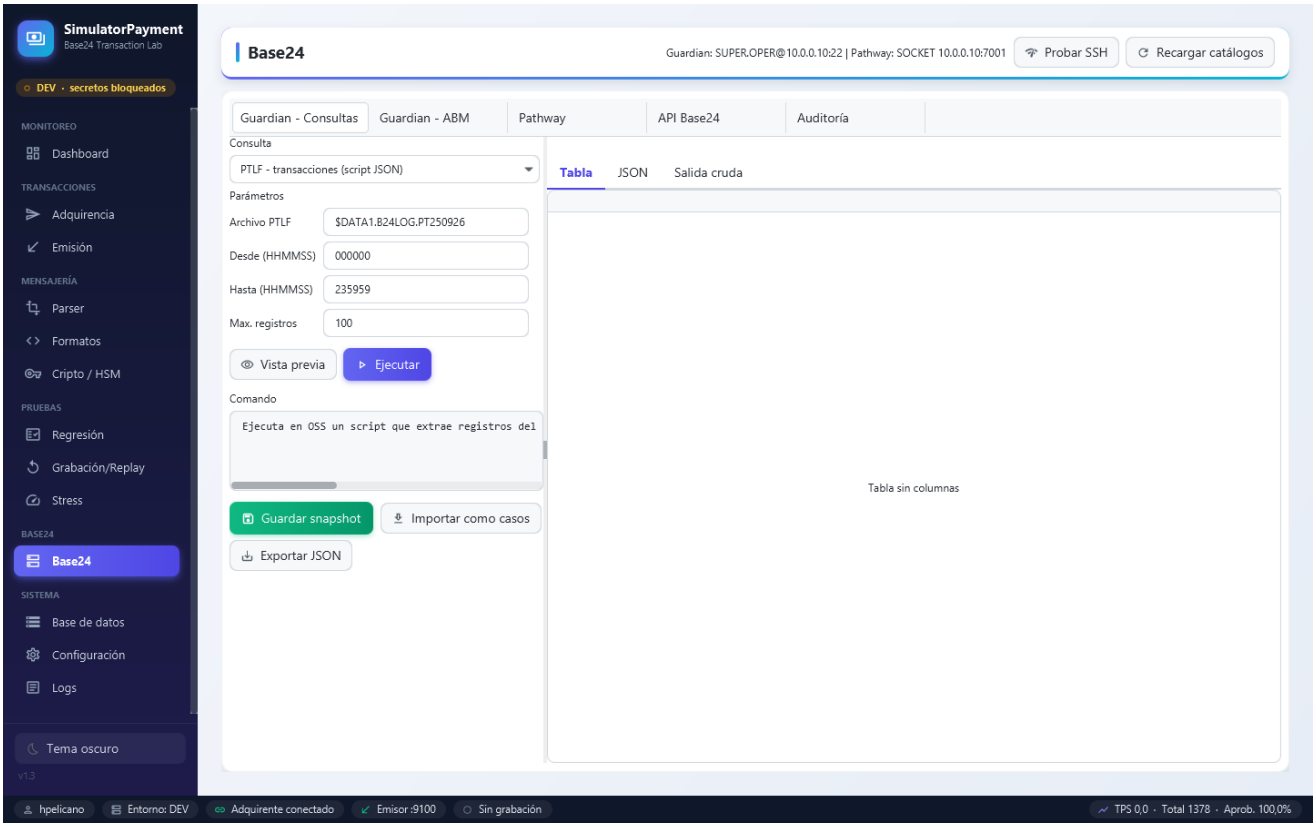


Figura 27. Herramientas de operación remota Base24.

Guardian consultas

- Catálogo parametrizado en `guardian-queries.json` para SQLCI, ENFORM o scripts.
- La salida puede presentarse como texto, JSON o tabla y queda asociada a la ejecución.
- SSH usa usuario/password o private key, known_hosts, StrictHostKeyChecking y timeouts configurables.

Guardian ABM

- El catálogo `guardian-abm.json` define operación, parámetros y comando.
- Ejecutar muestra primero un dry-run del comando, solicita confirmación y registra el resultado en auditoría.
- Los parámetros se validan contra regex y se rechazan saltos de línea o separadores peligrosos.

Pathway

- Modo SOCKET: conexión orientada a líneas con prompt y terminador configurados.
- Modo SSH: invoca PATHCOM por el canal SSH.
- allowedVerbs limita acciones; confirmVerbs exige confirmación para verbos críticos.
- Los comandos incluidos son ejemplos y deben adaptarse al sitio.

14. Base de datos H2

La base H2 embebida persiste mensajes reutilizables, dialectos, ejecuciones, auditoría y snapshots. Flyway administra el esquema; la versión cubierta es V6.

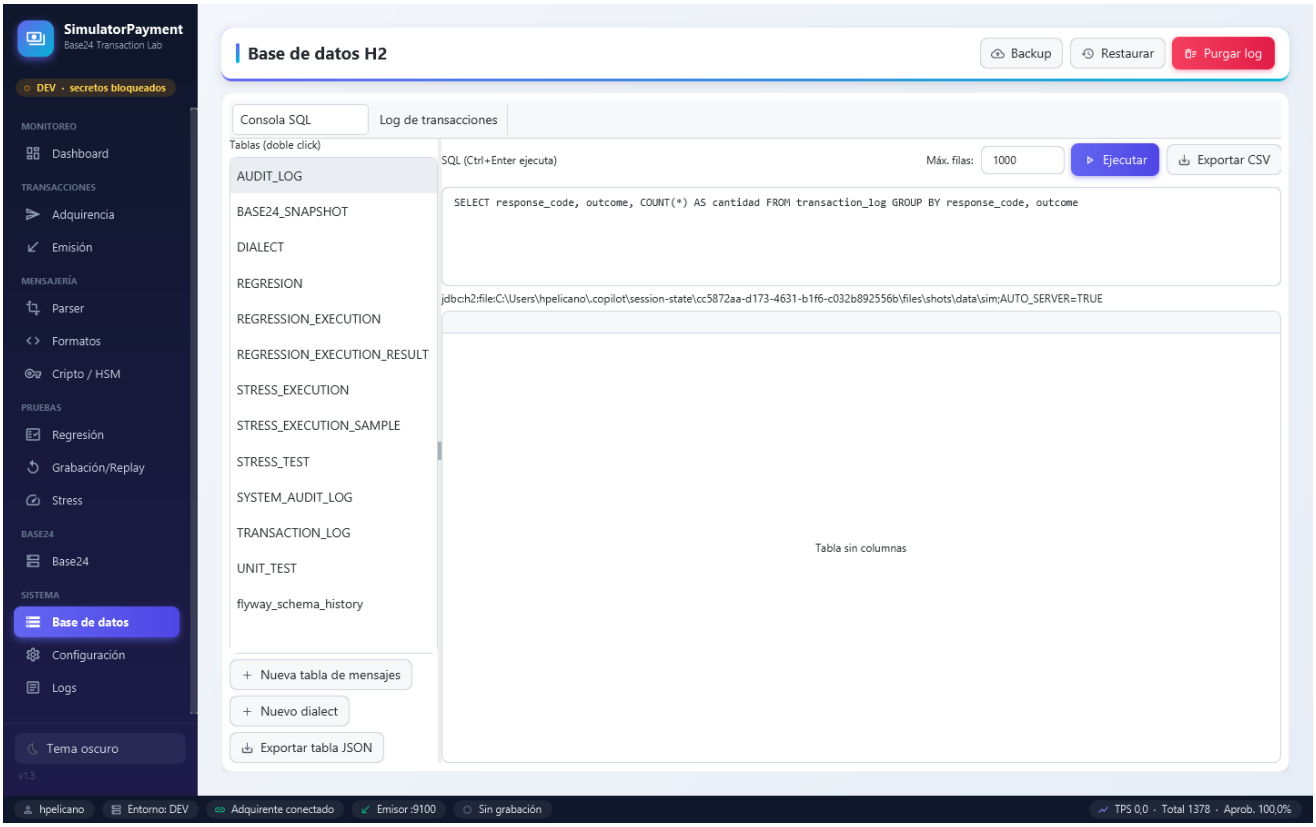


Figura 28. Administración y consulta de la base H2.

Tablas funcionales

Tabla	Propósito / columnas principales
dialect	id textual, descripción y timestamps; referencia de suite_id.
unit_test	id, suite_id, name, request, expected_rc, ignore_fields, enabled, created_at, updated_at.
regresion	Catálogo de transacciones para regresión con la misma estructura reutilizable.
stress_test	Mensajes base para carga con la misma estructura.
audit_log	id, request JSON, response JSON, timestamp y status.
system_audit_log	Auditoría de operaciones de sistema/operador.
transaction_log	Trazabilidad transaccional enmascarada, outcome y latencia.
run / run_case / snapshots	Corridas de regresión/stress, resultados y comparaciones.

Operación

- Permite inspeccionar tablas, ejecutar consultas administrativas y crear dialectos o tablas estándar de mensajes.
- Las tablas reutilizables deben contener suite_id, name, request, expected_rc, ignore_fields, enabled y timestamps.
- Save as y Update en Adquirencia/Emisión usan transacciones con commit/rollback explícito.
- No modifique una migración Flyway ya aplicada; cree la siguiente versión (V7 en adelante).

15. Configuración, ambientes y secretos

Configuración edita preferencias de usuario y los JSON de ambientes. Al guardar, valida puertos, timeouts, framing y referencias, y descarta clientes remotos para que se reconstruyan con los nuevos valores.

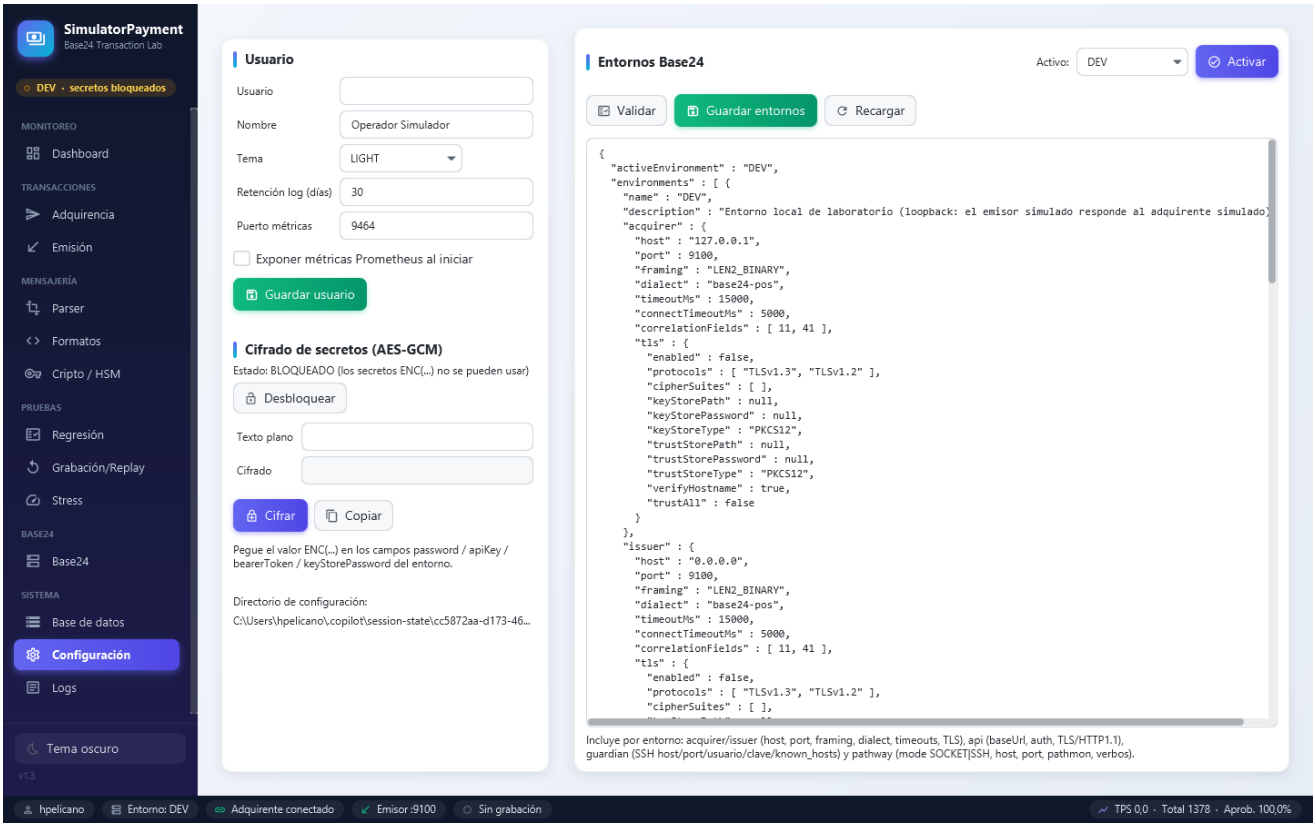


Figura 29. Configuración de usuario, ambientes, secretos y archivos.

Archivos

Archivo	Contenido
user.json	username, tema, puerto y habilitación de métricas.
environments.json	Ambientes DEV/QA/UAT y endpoints acquirer, issuer, issuerClient, issuerOutbound, API, Guardian, Pathway
issuer-rules.json	Reglas de respuesta de Emisión Server.
formats/*.json	Dialectos de usuario que agregan o reemplazan built-ins.
guardian-queries.json / guardian-abm.json	Catálogos operativos por sitio.

Secretos

- La UI cifra valores con AES-GCM; la clave se deriva por PBKDF2 desde una contraseña maestra de al menos 8 caracteres.
- Los valores se almacenan como ENC(...). La variable SIM_MASTER_PASSWORD permite desbloqueo no interactivo.
- El desbloqueo es diferido. Si se cancela un pedido implícito, no vuelve a insistir durante 15 segundos.
- No cifre host, puerto o dialecto; cifre passwords, API keys, bearer tokens y passwords de keystore/truststore.

15.1 Ejemplo de ambiente

El siguiente fragmento muestra los endpoints transaccionales esenciales. TLS y credenciales se omiten por brevedad.

```
{
  "name": "DEV",
  "acquirer": {
    "host": "127.0.0.1", "port": 9100,
    "framing": "LEN2_BINARY", "dialect": "base24-pos",
    "timeoutMs": 15000, "correlationFields": [11, 41]
  },
  "issuer": {
    "host": "0.0.0.0", "port": 9100,
    "framing": "LEN2_BINARY", "dialect": "base24-pos"
  },
  "issuerRouting": "FULL_DUPLEX",
  "issuerOutbound": {
    "host": "127.0.0.1", "port": 9101,
    "framing": "LEN2_BINARY", "dialect": "base24-pos"
  },
  "issuerClient": {
    "host": "127.0.0.1", "port": 9200,
    "framing": "LEN2_BINARY", "dialect": "base24-pos"
  }
}
```

TLS

- tls.enabled habilita SSLContext en cliente o servidor.
- protocols limita TLSv1.2/TLSv1.3; keyStore habilita identidad/mTLS y trustStore controla confianza.
- verifyHostname debe permanecer activo salvo laboratorios explícitos; trustAll no debe usarse en ambientes reales.
- El framing se aplica después de TLS: los bytes de longitud viajan dentro del canal cifrado.

16. Logs y diagnóstico

Logs muestra mensajes de la aplicación capturados por el appender UI: nivel, timestamp, logger, hilo y detalle. Es distinto del log transaccional de Emisión y de las tablas H2.

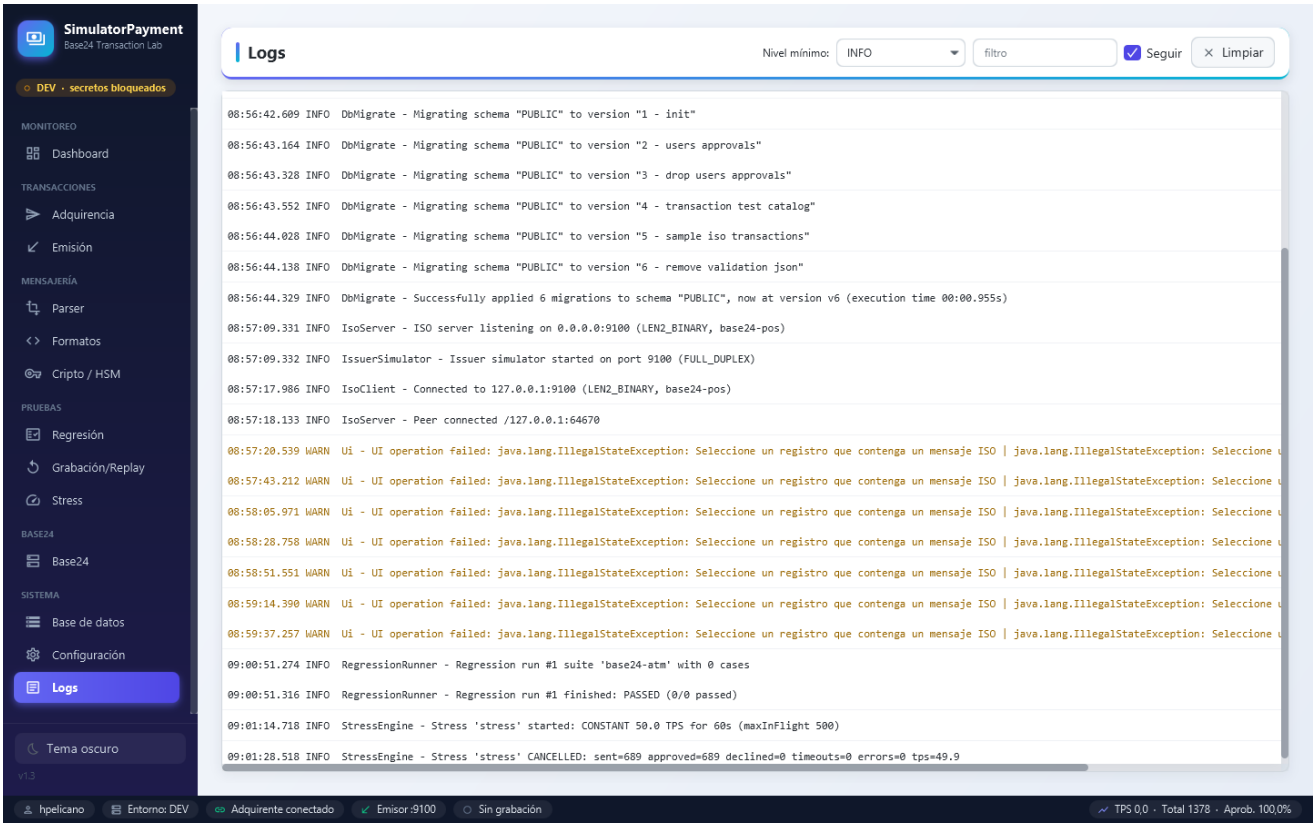


Figura 30. Consola de logs de la aplicación.

Qué buscar

- Cannot bind: puerto ocupado o IP local inválida.
- Cannot connect: host/puerto remoto inaccesible, firewall o TLS.
- Cannot decode request / Cannot pack response: dialecto, header, longitud o valor de campo incompatible.
- Timeout after …: no hubo respuesta correlacionada dentro de timeoutMs.
- Duplicate in-flight correlation key: DE11/DE41 repetidos mientras existe un request pendiente.
- Rule evaluation failed / Response post-processing failed: regla o cripto del emisor.
- Flyway validation: migración modificada o esquema inconsistente.

Secuencia de troubleshooting

- 1. Confirme ambiente activo, endpoint, DNS/IP y puerto.
- 2. Confirme framing en ambos lados; después dialecto.
- 3. Pruebe un 0800 o un mensaje mínimo conocido.
- 4. Use Parser/Trace sobre el payload exacto.
- 5. Revise correlación, timeout y STAN.
- 6. Revise TLS/SSH y secretos.
- 7. Exporte log de mensajes, log de aplicación y datos enmascarados del audit log.

17. Seguridad y cumplimiento

La aplicación incorpora controles preventivos, pero el operador conserva responsabilidad sobre claves, grabaciones, capturas, permisos del sistema operativo y conectividad.

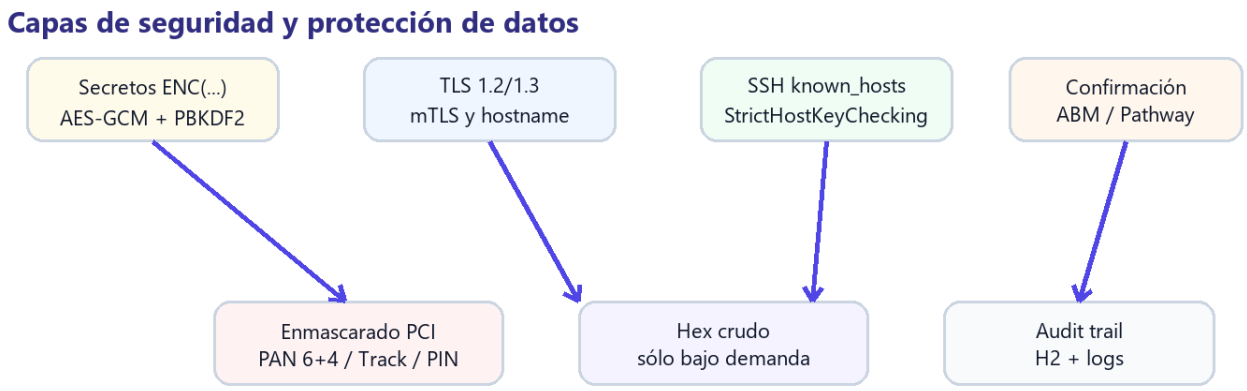


Figura 31. Capas de protección.

Controles incluidos

- PAN enmascarado 6+4, tracks enmascarados y DE52/DE53 eliminados antes de logs persistentes.
- El hex crudo no se persiste y requiere una acción explícita para mostrarse.
- Secretos cifrados AES-GCM con derivación PBKDF2.
- TLS 1.2/1.3, mTLS y verificación de hostname configurables.
- SSH con known_hosts y StrictHostKeyChecking.
- Confirmación de comandos críticos y audit trail.
- Validación anti-inyección de parámetros remotos.

No es PCI DSS por sí mismo. El enmascarado de la herramienta no certifica el entorno. Aplique controles de acceso, retención, segmentación, gestión de claves y borrado seguro de la organización.

18. Automatización headless y CI/CD

El fat-jar soporta comandos sin UI para regresión, emisor y agentes de stress. Esto permite integrar pruebas en pipelines.

```
# Regresión: exit 0 si pasa, 1 si falla
java -jar simulator-payment.jar --headless regression \
     --suite SMOKE --env QA --junit report.xml --html report.html

# Emisor standalone
java -jar simulator-payment.jar --headless issuer --port 9100

# Agente de stress
java -jar simulator-payment.jar --headless agent \
     --token S3cr3t0Agente --port 7700 --env QA

# Métricas Prometheus
java -jar simulator-payment.jar --headless issuer \
     --port 9100 --metrics 9477
```

Build

```
build.bat
build.bat skiptests
build.bat offline

mvn clean install
mvn -pl sim-app -am package
```

- El artefacto ejecutable queda en sim-app/target/simulator-payment.jar.
- Para CI, provea SIM_MASTER_PASSWORD por un secret store y no como texto en el repositorio.
- Publique HTML/JUnit como artefactos y conserve logs enmascarados.
- El modo headless usa la misma configuración y H2 que la UI salvo que se indiquen directorios alternativos.

19. Arquitectura técnica por módulo

El proyecto es un reactor Maven de 17 módulos. La separación evita acoplar la UI con codecs, transporte o persistencia.

Módulo	Responsabilidad técnica
sim-common	JSON/Jackson, EventBus, TransactionEvent, Hex/BCD/EBCDIC/Luhn y masking PCI.
sim-config	Modelos de config, cifrado de secretos, validación y SSLContext.
sim-persistence	H2, Flyway, HikariCP, repositorios y DbAdminService.
sim-iso-parser	DialectRegistry, MessageService, codecs, FieldTrace y conversión.
sim-validator	Perfiles, reglas y ValidationResult/ValidationIssue.
sim-transport	IsoClient/IsoServer Netty, framing, TLS, correlación y timeout.
sim-acquirer	AcquirerService, templates, secuencias, reversos y outcomes.
sim-issuer	IssuerSimulator, reglas, ruteo full/split y observadores de tráfico.
sim-crypto	PIN, MAC, CVV, EMV y hooks transaccionales.
sim-regression	Runner, reportes, comparación, recorder y replay.
sim-stress	Motor de carga, perfiles, SLA, coordinador/agentes.
sim-stats	Agregación, HdrHistogram y Prometheus.
sim-ssh / sim-base24	SSH/SFTP, Guardian, ABM y Pathway.
sim-api-client	HTTP/TLS/auth/retry para gateway Base24.
sim-ui / sim-app	JavaFX, wiring AppContext, launcher, fat-jar y headless.

19.1 Flujo interno de mensajería

MessageService es la frontera entre la representación lógica y el wire. IsoClient/IsoServer nunca interpretan campos: delegan pack/unpack al servicio con el dialecto del endpoint.

```
JSON --fromJson--> IsoMessage --codec(dialect).pack--> payload ISO8583
payload --Framing.frame--> stream TCP
stream TCP --frameDecoder--> payload
payload --unpack(dialect)--> IsoMessage --toJson--> JSON
```

Correlación

- IsoClient admite múltiples requests en vuelo sobre una conexión.
- La clave se construye con correlationFields; por defecto DE11 + DE41.
- Una respuesta elimina el pending correspondiente y completa Exchange con request/response, raw bytes y latencia.
- Un timeout programado elimina el pending y completa excepcionalmente con TransportException(timeout=true).
- Requests iniciados por el host pueden procesarse con requestHandler; respuestas no correlacionadas llegan como unsolicited.

Eventos y persistencia

- AcquirerService convierte Exchange en TransactionOutcome, valida, publica TransactionEvent y escribe transaction_log.
- IssuerSimulator publica events, notifica Traffic observers y devuelve Reply o enruta por puerto 2.
- StatisticsCollector consume events; TrafficRecorder consume outcomes/traffic cuando está grabando.

20. Operación recomendada por escenario

Esta matriz resume el módulo indicado según el objetivo.

Objetivo	Módulos	Secuencia
Validar una compra	Emisión Server + Adquirencia	Iniciar emisor → conectar adquirente → seleccionar caso → Send → Validación/Trace.
Integrar contra Base24 real	Configuración + Adquirencia	Definir endpoint/TLS/dialecto/framing → Echo → caso controlado → auditoría.
Simular Visa/MC iniciando tráfico	Emisión Cliente	Configurar issuerClient → seleccionar request H2 → conectar → Send.
Probar host con puertos separados	Emisión Server	Definir P1/P2 y framing → SPLIT_LISTEN o SPLIT_CONNECT → revisar log ERR.
Diagnosticar wire	Parser + Formatos	Retirar framing → Hex→JSON → Trace → comparar definición.
Certificar release	Regresión + Replay	Suite baseline → ejecutar → comparar corridas → reportes HTML/JUnit.
Medir capacidad	Stress + Dashboard	Warm-up → perfil estable → SLA → CSV/Prometheus.
Operar Base24	Base24 + Logs	Configurar SSH/Pathway → dry-run → confirmar → auditar.

21. Checklist de preparación de ambiente

Complete esta lista antes de una campaña formal.

□	Control
□	Ambiente activo correcto; hosts y puertos aprobados por infraestructura.
□	Dialecto validado con vectores oficiales y round-trip.
□	Framing validado en ambos sentidos; P1/P2 si aplica.
□	Correlation fields compatibles y STAN/RRN no duplicados.
□	TLS/mTLS, truststore, hostname y fechas de certificados validados.
□	Secretos ENC(...), known_hosts y claves de laboratorio controlados.
□	Reglas del emisor revisadas; noResponse/delay intencionales documentados.
□	Datos H2 y expected_rc/ignore_fields revisados.
□	Dashboard reiniciado; grabación/log habilitado según necesidad.
□	Criterios de éxito, SLA y reportes definidos.
□	Plan de reverso/limpieza y responsables de incidentes disponibles.

Checklist de cierre

- Detener Stress, Emisor, grabación y conexiones.
- Exportar reportes, logs y estadísticas; verificar enmascarado.
- Guardar configuración/reglas/formats versionados sin secretos.
- Retirar grabaciones sensibles y datos temporales según política.
- Documentar defectos con request/response enmascarados, trace, framing, dialecto y timestamp.

22. Limitaciones y supuestos

Conocer los límites evita interpretar un resultado de laboratorio como una garantía de producción.

- BNET y HPDH son plantillas: longitudes y encodings deben ajustarse a la especificación real.
- Los comandos Guardian/Pathway incluidos son ejemplos y no describen todas las instalaciones.
- NONE no puede reconstruir mensajes fragmentados o múltiples mensajes coalescidos por TCP.
- La conversión entre dialectos copia campos compatibles; no transforma automáticamente semántica de marcas.
- El HSM de software no es un dispositivo certificado ni protege claves como hardware.
- Los percentiles distribuidos usan una aproximación conservadora.
- El simulador es monousuario; la autorización y segregación dependen del sistema operativo y del proceso organizacional.

Compatibilidad y cambios

- No edite migraciones Flyway ya aplicadas; agregue versiones.
- Al agregar un atributo de regla, actualice IssuerRuleEngine, RulesEditor y pruebas.
- Al agregar framing, actualice enum/decoder/encoder, validación config, FramingChoice, documentación y TransportTest.
- Al agregar un dialecto, pruebe pack/unpack/trace y marque campos sensibles.
- Actualice el SKILL.md del módulo modificado para conservar memoria técnica.

23. Glosario

Términos usados en la aplicación y el manual.

Término	Definición
Dialect / dialecto	Identificador de una definición de mensajería usada por el codec.
suite_id	Clasificador H2 de casos; puede coincidir o no con un dialecto real.
Framing	Prefijo/estrategia que delimita mensajes dentro del stream TCP.
MTI	Message Type Indicator ISO8583 (por ejemplo 0200 request, 0210 response).
DE	Data Element/campo ISO8583; DE39 es response code, DE11 STAN.
STAN	Systems Trace Audit Number; identificador transaccional de seis dígitos habitual.
RRN	Retrieval Reference Number (DE37).
RC	Response code (DE39).
Echo fields	Campos que se esperan iguales entre request y response.
Outcome	APPROVED, DECLINED, TIMEOUT o ERROR.
Full-duplex	Request y response comparten la misma conexión bidireccional.
Split routing	Entrada por un enlace y respuesta por otro puerto/enlace.
ARQC / ARPC	Criptograma EMV de request / respuesta.
KCV	Key Check Value para identificar una clave sin revelarla.
TPS	Transacciones por segundo.
p95 / p99	Percentil de latencia por debajo del cual cae el 95% / 99% de muestras.

Apéndice A. Referencia exhaustiva de controles UI

Esta referencia enumera los controles que pueden variar levemente según datos o configuración.

1) DashboardView

Título sidebar: `Dashboard`

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/DashboardView.java`

Layout y controles

- Barra superior: `Estadísticas de transacciones`
- Botones:
- `Reiniciar`
- `Exportar CSV`
- Toggle `Prometheus: OFF` o `Prometheus: http://localhost:/metrics`
- KPIs mostrados:
- `Total`, `Aprobadas`, `Rechazadas`, `Timeouts`, `Errores`
- `% Aprob.`
- `TPS actual`, `TPS pico`, `TPS prom.`
- `P50 ms`, `P95 ms`, `P99 ms`, `Máx ms`
- Gráficos:
- Línea: `TPS (últimos 5 minutos)` con ejes `segundos` / `TPS`
- Pie: `Resultado`
- Tablas:
- `Por código de respuesta` con columnas: código y `Cantidad`
- `Por origen` con columnas: origen y `Cantidad`

Acciones

- `Reiniciar`: llama `ctx.stats().reset()` y refresca.
- `Exportar CSV`: descarga `ctx.stats().toCsv()` como `estadisticas.csv`.
- Toggle Prometheus: arranca/detiene `ctx.startMetrics()` / `ctx.stopMetrics()`.

Configuración relacionada

- Exporter Prometheus del `AppContext`.
- Dependiente de `sim-stats`.

Validaciones / límites

- Refresco automático cada 1 s.
- Historial TPS tomado de `StatsSnapshot`.
- Si la vista no está en escena, `refresh()` no hace nada.

---

2) ParserView

Título sidebar: `Parser`

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/ParserView.java`

Layout y controles

- Barra superior: `Parser de mensajería`
- Controles:
- `Formato:` + ChoiceBox de dialecto
- Botones: `JSON → Hex`, `Hex → JSON`, `Trace`, `Validar estructura`, `Convertir`, `Formatear JSON`
- `Convertir a:` + ChoiceBox destino
- Editores:
- `JSON`
- `Wire (hex)`
- Tabs:
- `Trace`
- `Validación`
- `Definición del formato`
- Tabla de trace con columnas:
- `Elemento`, `Nombre`, `Offset`, `Long.`, `Valor`, `Hex`
- Área de issues y definición.
- Mensaje informativo arriba del split.

Acciones

- `JSON → Hex`: parsea JSON, empaqueta con `ctx.messages().pack`, muestra hex, actualiza trace.
- `Hex → JSON`: elimina no-hex, desempaqueta con dialecto actual.
- `Trace`: traza desde JSON o hex.
- `Validar estructura`: valida con `ctx.validator().validateStructure("message", ...)`.
- `Convertir`: convierte a dialecto destino con `ctx.messages().convert`.
- `Formatear JSON`: pretty-print del JSON.
- Cambio de dialecto: actualiza definición del formato.

Configuración relacionada

- Registro de dialectos de `ctx.messages().registry()`.
- Validador `sim-validator`.

Validaciones / límites

- Si el JSON es inválido, `Formatear JSON` muestra error.
- `Hex → JSON` tolera espacios y separadores no hex.
- `Validate` usa dialecto actual del mensaje.
- `Trace` funciona incluso si hex está vacío, empaquetando primero.

---

3) FormatsView

Título sidebar: `Formatos`

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/FormatsView.java`

Layout y controles

- Barra superior: `Editor de formatos`
- `Dialecto:` ChoiceBox + botón `Recargar`
- Formulario:
- `Id (guardar como)`
- `Nombre`
- `Descripción`
- Encodings:
- `MTI`, `Bitmap`, `Numérico`, `Texto`, `Largo`
- Tabla editable con columnas:
- `Campo`
- `Nombre`
- `Tipo`
- `Longitud`
- `Máx.`
- `Encoding`
- `Enc. largo`
- `Sub-formato`
- `Pad`
- `Sensible`
- Botones:
- `Agregar campo`
- `Quitar campo`
- `Ver JSON`
- `Guardar y aplicar`
- Nota: se guarda en `config/formats/.json`.

Acciones

- `Recargar`: repuebla lista desde registry.
- `Agregar campo`: pide número 2–128; crea `FieldDef` por defecto.
- `Quitar campo`: elimina campo seleccionado.
- `Ver JSON`: guarda/exporta JSON del dialecto construido.
- `Guardar y aplicar`:
- valida id con regex `[A-Za-z0-9._-]{1,60}`
- escribe archivo
- registra dialecto

- resuelve herencias
- limpia caché

Configuración relacionada

- `config/formats/.json`
- `ctx.messages().registry()`
- `ctx.messages().clearCache()`

Validaciones / límites

- No permite id inválido.
- No permite número duplicado o fuera de 2–128.
- Pregunta antes de sobrescribir archivo existente.

---

4) CryptoView

Título sidebar: `Cripto / HSM`

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/CryptoView.java`

Layout y controles

- Título: `Criptografía (HSM de software)`
- Tab `Configuración`
- aviso: `Solo claves de TEST. Nunca cargar claves productivas...`
- checkbox `Habilitar HSM de software en adquirente/emisor`
- campos:
- `ZPK (hex o ENC(...))`
- `MAK (hex o ENC(...))`
- `CVK A||B (hex o ENC(...))`
- `IMK-AC (hex o ENC(...))`
- `Algoritmo MAC`
- `Formato PIN block`
- `Session key EMV`
- `IAD en ARQC`
- `RC si ARQC inválido (vacío = no cambiar)`
- checkboxes:
- `Generar ARQC (9F26) en requests`
- `Generar ARPC (tag 91) en respuestas del emisor`
- `Verificar ARQC en el emisor`
- `Cifrar claves al guardar (ENC)`
- botones: `Guardar`, `Recargar`, `KCV de claves`
- Tab `Calculadoras`
- PIN block:
- `PIN`, `PAN`, `Bloque`
- botones `Cifrar`, `Descifrar`
- CVV:
- `Venc.`, `Service`, botón `Calcular`
- MAC:
- `Datos (texto o hex:...)`, botón `Calcular MAC`
- Pipeline mensaje:
- text area JSON
- botones `Aplicar (adquirente)`, `Verificar (emisor)`, `Tags EMV`
- Salida abajo.

Acciones

- `Guardar`: persiste `CryptoConfig` en el entorno activo.

- `Recargar`: recarga config.
- `KCV de claves`: muestra KCV de `zpk`, `mak`, `cvk`, `imkAc`.
- Calculadoras usan `PinBlock`, `Cvv`, `Mac`, `Emv`.
- `Aplicar (adquirente)`: fuerza crypto enabled y ejecuta `prepareRequest`.
- `Verificar (emisor)`: valida ARQC, MAC y PIN si presentes.
- `Tags EMV`: pretty-print de tags DE55.

Configuración relacionada

- `CryptoConfig` del entorno activo.
- Cifrado secreto vía `ctx.config().encryptSecret`.
- Claves leídas con `ctx.crypto().key(...)`.

Validaciones / límites

- Solo claves de test.
- `protect()` no cifra si ya empieza con `ENC(` o si no está desbloqueado.
- `Guardar` persiste `ENC(...)` si aplica.
- `Verificar` no rompe si faltan campos.

---

5) RegressionView

Título sidebar: `Regresión`

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/RegressionView.java`

Layout y controles

- Barra superior:
- `Ejecutar suite`
- `Cancelar`
- `Exportar HTML`
- `Exportar JUnit XML`
- Parámetros de ejecución:
- `Concurrencia:`
- `Pausa ms:`
- `Ignorar campos:`
- `tag` (filtro, sin label visible salvo campo `tags`)
- checkbox `Refrescar campos dinámicos (STAN/RRN/fechas)`
- checkbox `Detener en primer fallo`
- Barra de progreso y resumen.
- Izquierda: `Suites`
- botones `Nueva`, `Eliminar`, `Refrescar`
- Tabs:
- `Casos`
- `Resultados`
- `Historial`
- Tabla de casos:
- `Id`, `Nombre`, `Formato`, `RC esp.`, `Origen`, `Tags`, `Activo`
- Editor de caso:
- `RC esperado:`
- botones `Guardar caso`, `Nuevo caso desde JSON`, `Eliminar caso`
- JSON del request
- Tabla resultados:
- `Caso`, `Resultado`, `RC esp.`, `RC real`, `Outcome`, `Latencia ms`, `Detalle`
- Historial:
- `Run`, `Suite`, `Entorno`, `Inicio`, `Total`, `OK`, `Fallas`, `Estado`
- botón `Comparar 2 corridas`
- nota de ignorados dinámicos: `7, 11, 12, 13, 15, 17, 37, 38`

Acciones

- `Ejecutar suite`: valida suite, conexión de adquirencia, opciones; ejecuta `RegressionRunner`.
- `Cancelar`: llama `runner.cancel()`.

- Exportaciones generan HTML o JUnit XML.
- `Nueva` suite: prompt nombre.
- `Eliminar` suite: confirma.
- `Guardar caso`: persiste el caso actual.
- `Nuevo caso desde JSON`: crea caso en suite con RC por defecto `00`.
- `Eliminar caso`: confirma y borra.
- `Comparar 2 corridas`: usa `RegressionComparator`, puede guardar HTML.

Configuración relacionada

- H2/persistencia (`TestCaseRepository`, `RunRepository`).
- `RegressionOptions`.

Validaciones / límites

- No ejecuta sin suite o sin canal de adquirencia conectado.
- No permite segunda ejecución simultánea.
- `ignore` parsea enteros por coma/espacio.
- `tagFilter` opcional.
- Resalta filas fallidas.

---

6) ReplayView

Título sidebar: Grabación/Replay

Archivo: sim-ui/src/main/java/org/simulator/ui/view/ReplayView.java

Layout y controles

- Sección Grabación de tráfico
- Nombre: + campo
- botones ● Grabar, ■ Detener, Guardar (cifrado), Abrir
- estado: GRABANDO, Sin grabación, o : tx
- tabla items grabados:
- Offset ms, Origen, MTI, RC, Estado, Latencia ms
- Sección Replay
- Velocidad:
- Orígenes:
- checkbox Regenerar STAN/fechas/RRN
- botones Reproducir, Cancelar
- barra de progreso
- resumen
- tabla resultados:
- #, Origen, MTI, STAN, RC orig., RC replay, OK, Estado, Error
- Botón: Guardar reporte HTML

Acciones

- ● Grabar: inicia ctx.recorder().start(...)
- ■ Detener: detiene y carga la grabación.
- Guardar (cifrado): exige configuración desbloqueada, guarda .simrec cifrado.
- Abrir: exige desbloqueo, carga .simrec.
- Reproducir: filtra por orígenes, aplica velocidad, regeneración opcional, ejecuta ReplayEngine.
- Cancelar: cancela engine.
- Guardar reporte HTML: exporta lastReport.toHtml().

Configuración relacionada

- RecordingStore.save/load con config::encryptSecret / config::secret.
- Requiere master password desbloqueada.

Validaciones / límites

- No guarda/abre si configuración bloqueada.
- No reproduce sin grabación.
- Requiere adquirente conectado.
- Velocidad numérica válida.
- Orígenes se interpretan como CSV y se normalizan a mayúsculas.

---

7) StressView

Título sidebar: `Stress`

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/StressView.java`

Layout y controles

- Barra superior: `Stress test (TPS)` + tip.
- Formulario:
- `Nombre`
- `Plantilla`
- `Perfil`
- `TPS objetivo`
- `TPS inicial (RAMP/STEP)`
- `Duración (s)`
- `Rampa (s)`
- `Incremento STEP (TPS)`
- `Cada (s)`
- `TPS pico (SPIKE)`
- `Pico en (s)`
- `Duración pico (s)`
- `Máx. en vuelo`
- `SLA P99 máx ms (0=off)`
- `SLA % error máx (0=off)`
- `SLA gracia (s)`
- Botones:
- `Iniciar`
- `Detener`
- `Exportar CSV`
- Sección `Distribuido (agentes)`
- text area con formato `host:port@token`
- botones `Health`, `Ejecutar distribuido`
- `Resultado`
- Gráficos:
- `Throughput (TPS)` con series `TPS objetivo`, `TPS logrado`, `Errores+timeouts/s`
- `Latencia` con `P50 ms`, `P99 ms`
- Label live con estado del segundo actual.

Acciones

- `Iniciar`: valida plantilla, conexión, parsea perfil y SLA, ejecuta `StressEngine`.
- `Detener`: detiene engine y coordinador.
- `Exportar CSV`: guarda último resultado.

- `Health`: consulta salud de agentes.
- `Ejecutar distribuido`: construye `AgentRequest`, reparte carga entre agentes, lanza `StressCoordinator`.
- Cada muestra actualiza charts y texto live.
- Finaliza guardando corrida y samples en repositorio.

Configuración relacionada

- `LoadProfile`
- `SlaPolicy`
- Repositorios `ctx.runs()`
- `StressCoordinator`, `AgentRequest`

Validaciones / límites

- No inicia si ya corre otra prueba.
- No inicia sin plantilla.
- Requiere adquirente conectado.
- Numéricos parseados con `.` o `,` para TPS.
- SLA 0 desactiva umbral.
- `maxInFlight` controla throttling.
- `dispose()` detiene motor.

---

8) Base24View

Título sidebar: `Base24`

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/Base24View.java`

Layout general

Tabs:

- `Guardian - Consultas`
- `Guardian - ABM`
- `Pathway`
- `API Base24`
- `Auditoría`

Barra superior:

- botón `Probar SSH`
- botón `Recargar catálogos`
- label de estado SSH

Guardian - Consultas

- `Consulta` + ChoiceBox
- `Parámetros` + formulario dinámico
- botones `Vista previa`, `Ejecutar`
- `Comando`
- botones `Guardar snapshot`, `Importar como casos`, `Exportar JSON`
- panel derecho con tabs:
- `Tabla`
- `JSON`
- `Salida cruda`
- label `queryInfo`

Guardian - ABM

- `Operación ABM` + ChoiceBox
- `Parámetros`
- botones `Simular (dry-run)`, `Ejecutar`
- `Comando`, `Salida`
- nota: ejecución directa tras confirmación

Pathway

- `Comando PATHCOM:`
- botón `Enviar`
- botón `Desconectar`
- botones rápidos:
- `STATUS PATHMON`
- `INFO PATHWAY`

- `STATUS SERVER *`
- `STATUS TCP *`
- `STATUS TERM *`
- historial a la derecha
- salida y estado

API Base24

- `Método` GET/POST/PUT/DELETE
- `Path`
- botón `Enviar`
- `Request`
- `Response`

Auditoría

- tabla columnas:
- `Fecha`, `Usuario`, `Acción`, `Objetivo`, `Detalle`, `OK`
- botón `Refrescar`

Acciones

- Consultas:
- `Vista previa` renderiza comando.
- `Ejecutar` corre consulta SSH, llena tabla/JSON/raw.
- `Guardar snapshot` persiste en H2.
- `Importar como casos` crea casos de regresión.
- ABM:
- `Simular` arma comando sin ejecutar.
- `Ejecutar` muestra confirmación y luego corre.
- Pathway:
- valida política `CommandPolicy`
- puede exigir confirmación
- guarda historial y auditoría
- API:
- hace request HTTP/1.1 y muestra status, headers, body pretty.
- Auditoría:
- carga `auditTrail(500)`.

Configuración relacionada

- Catálogos `guardian-queries.json`, `guardian-abm.json`.
- `ctx.ssh()`, `ctx.pathway()`, `ctx.api()`.
- `ctx.env()` para host/port/path.
- `CommandPolicy` para whitelist.

Validaciones / límites

- `CommandPolicy` bloquea comandos denegados y encadenamiento.
- ABM exige confirmación si el template lo requiere o por acción.
- API usa path por defecto del entorno si está vacío.
- Auditoría muestra hasta 500.

---

9) DatabaseView

Título sidebar: `Base de datos`

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/DatabaseView.java`

Layout

- Barra superior: `Base de datos H2`
- Botones: `Backup`, `Restaurar`, `Purgar log`
- Tabs:
- `Consola SQL`
- `Log de transacciones`

Consola SQL

- izquierda:
- `Tablas (doble click)`
- botones `Nueva tabla de mensajes`, `Nuevo dialect`, `Exportar tabla JSON`
- derecha:
- `SQL (Ctrl+Enter ejecuta)`
- `Máx. filas:`
- botones `Ejecutar`, `Exportar CSV`
- resultado en tabla
- info de filas/tiempo
- doble clic en tabla inserta `SELECT * FROM ORDER BY 1 DESC LIMIT 200`

Log de transacciones

- botón `Refrescar`
- nota: `(últimas 1000, datos sensibles enmascarados)`
- tabla con columnas:
- `Id`, `Fecha`, `Origen`, `Entorno`, `MTI`, `PC`, `STAN`, `RRN`, `PAN`, `Importe`, `RC`, `Outcome`, `Latencia ms`, `Error`
- detalle request/response abajo

Acciones

- `Ejecutar`: corre SQL vía `ctx.dbAdmin().execute`.
- `Exportar CSV`: exporta la grilla actual.
- `Exportar tabla JSON`: exporta tabla seleccionada.
- `Nueva tabla de mensajes`: crea tabla estándar validando nombre.
- `Nuevo dialect`: crea registro dialect.
- `Backup`: guarda ZIP o SQL.
- `Restaurar`: reemplaza base tras confirmación.
- `Purgar log`: elimina por antigüedad.

Configuración relacionada

- `ctx.dbAdmin()`, `ctx.txLog()`
- H2 local.

- Retención desde `ctx.config().user().getLogRetentionDays()`.

Validaciones / límites

- `DROP/DELETE/TRUNCATE` pide confirmación.
- Nombre de tabla: `[A-Za-z_][A-Za-z0-9_]*`
- `maxRows` inválido cae a 1000.
- Restaurar confirma reemplazo total.

---

10) ConfigView

Título sidebar: `Configuración`

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/ConfigView.java`

Layout

- Izquierda `Usuario`
- campos:
- `Usuario`
- `Nombre`
- `Tema`
- `Retención log (días)`
- `Puerto métricas`
- checkbox `Exponer métricas Prometheus al iniciar`
- botón `Guardar usuario`
- Sección `Cifrado de secretos (AES-GCM)`
- estado lock: `desbloqueado` o `BLOQUEADO`
- botón `Desbloquear`
- campos:
- `Texto plano`
- `Cifrado`
- botones `Cifrar`, `Copiar`
- texto de ayuda para pegar `ENC(...)` en `password / apiKey / bearerToken / keyStorePassword`
- directorio de configuración
- Derecha `Entornos Base24`
- `Activo:` + ChoiceBox
- botones `Activar`, `Validar`, `Guardar entornos`, `Recargar`
- editor JSON del `AppConfig`
- nota con contenido por entorno: `acquirer/issuer`, `api`, `guardian`, `pathway`

Acciones

- `Guardar usuario`: persiste usuario, tema, retención y métricas.
- `Desbloquear`: abre diálogo de master password.
- `Cifrar`: cifra texto plano con master key.
- `Copiar`: copia el valor ENC al clipboard.
- `Validar`: valida JSON y configuración sin tocar la real.
- `Guardar entornos`: escribe archivo de entornos, recarga configuración y notifica reconexión.
- `Activar`: cambia entorno activo, guarda y reconecta.

Configuración relacionada

- `ConfigService`, `AppConfig`, `EnvironmentConfig`
- `user.json` / archivos de entorno

- directorio `ctx.config().getConfigDir()`

Validaciones / límites

- Validación sobre copia temporal.
- `Guardar entornos` aborta si hay errores.
- `Activar` requiere nombre no nulo.
- `Cifrar` exige configuración desbloqueada.

---

11) LogsView

Título sidebar: `Logs`

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/LogsView.java`

Layout

- Barra superior: `Logs`
- `Nivel mínimo:` ChoiceBox `TRACE|DEBUG|INFO|WARN|ERROR`
- filtro text
- checkbox `Seguir`
- botón `Limpiar`
- lista de logs monoespaciada

Acciones

- Filtra por nivel y texto.
- `Limpiar`: vacía appender y lista.
- auto-drain cada 400 ms.
- `Seguir`: autoscroll al final.

Configuración relacionada

- `UiLogAppender`

Validaciones / límites

- Máximo 5000 entradas en memoria.
- Colorea `ERROR` y `WARN`.

---

12) RulesEditor

Archivo: `sim-ui/src/main/java/org/simulator/ui/view/RulesEditor.java`

Propósito: editor de reglas de respuesta del emisor.

Layout

- Lista de reglas a la izquierda.
- Botones: +, -, ↑, ↓
- Formularios:
- `RC por defecto`
- `Demora por defecto ms`
- checkbox `Generar código de autorización (DE38)`
- `Regla seleccionada`
- `Nombre`
- checkbox `Habilitada`
- condición:
- `MTI`
- `Processing code`
- `BIN prefijos`
- `Importe mín.`
- `Importe máx.`
- `Terminal`
- `Campos`
- acción:
- `RC`
- `Secuencia RC`
- `Demora ms`
- `Jitter ms`
- `Setear campos`
- `Quitar campos`
- checkbox `Sin respuesta (forzar timeout)`

Acciones

- +: agrega regla nueva con RC `00`.
- -: elimina seleccionada.
- ↑/↓: reordena.
- Al cambiar de selección, persiste cambios previos.
- `getRules()` arma `IssuerRules`.

Validaciones / límites

- Campos numéricos lanzan error si inválidos.
- `matchFields` y `setFields` usan `;` como separador, = entre clave/valor.

- `removeFields` acepta CSV de números.

Apéndice B. Referencia backend y extensibilidad

sim-stats

- `StatisticsCollector`: `attach(EventBus)`, `record(e)`, `snapshot()`, `reset()`, `toCsv()`.
- `PrometheusExporter`: `attach(bus)`, `start(port)`, `stop`, `isRunning`, `port`, `scrape()`, `registerGauge(...)`.
- Métricas: transacciones totales, response codes, por MTI, histograma latencia, gauges de TPS, aprobación, latencias, conexión adquirente, conexiones issuer, stress running.
- Formato Prometheus v0.0.4.
- Labels escapadas, decimales con `Locale.ROOT`.

sim-regression

- `RegressionOptions`: `concurrency`, `refreshDynamicFields` (true), `stopOnFirstFailure`, `delayBetweenMs`, `ignoreFields`, `tagFilter`.
- `RegressionRunner` ejecuta suite contra adquirencia.
- `CaseResult`: nombre, passed, RC esperado/real, issues, request/response JSON, error, latencia.
- `RegressionReport`: `passed`, `failed`, `status`, `toHtml()`, `toJUnitXml()`.
- `compare`: `RegressionComparator`, `ComparisonReport` con cambios `SAME`, `REGRESSION`, `FIXED`, `RC_CHANGED`, `FIELDS_CHANGED`, `ONLY_IN_BASE`, `ONLY_IN_OTHER`.
- `replay`: `Recording`, `RecordedTransaction`, `RecordingStore.save/load`, `ReplayEngine`, `ReplayReport`.

sim-stress

- `LoadProfile.Type`: `CONSTANT`, `RAMP`, `STEP`, `SPIKE`.
- Parámetros: `targetTps`, `startTps`, `durationSec`, `rampSec`, `stepTps`, `stepEverySec`, `spikeTps`, `spikeAtSec`, `spikeDurationSec`, `maxInFlight`.
- `StressEngine.run(...)` bloquea; `stop()`, `isRunning()`.
- `StressTarget` desacopla destino.
- Distribuido:
- `StressAgentServer`
- `AgentRequest`
- `AgentStatus`
- `StressCoordinator`
- `SlaPolicy`: umbrales de P99 y error rate.

sim-base24

- Catálogos:
- `guardian-queries.json`: ids ejemplo `ptlf-json`, `sqlci-txn`, `caf-card`
- `guardian-abm.json`: ids `caf-status`, `caf-insert`, `caf-delete`
- `TemplateCatalog.load(...)` mezcla classpath + usuario.
- `CommandRenderer.render(...)` sustituye `${param}`.
- `GuardianQueryService` ejecuta consultas SSH y parsea salida.
- `GuardianAbmService` ejecuta ABM; ya no hay maker-checker.
- `CommandPolicy`: verdict `ALLOWED`, `NEEDS_CONFIRMATION`, `DENIED`; bloquea `;` y saltos de línea.
- `PathwayService`: ejecuta y audita.

sim-ssh

- `SshClient`:
- `testConnection()`
- `exec(cmd[, timeout])`
- `openShell(lineTerminator)`
- SFTP: `upload`, `download`, `readRemoteFile`
- Autenticación por password o clave privada; ambos leen `secrets`, aceptan `ENC(...)`.
- `strictHostKeyChecking` + `knownHostsPath`.
- Timeouts configurables en `SshEndpoint`.

sim-api-client

- `Base24ApiClient(ApiEndpoint, secrets, MessageService)`
- Usa `HttpClient` forzado a `HTTP_1_1`.
- `ApiResponse`: status, body, `IsoMessage response`, latencia.
- Soporta auth/TLS según `ApiEndpoint`.

sim-crypto

- Primitivas JCE estándar DES/DESede/AES.
- `CryptoService.prepareRequest(req)`:
- reemplaza `DE52=PIN:` por PIN block cifrado
- recalcula ARQC si `DE55` contiene `9F26` y `9F36`
- calcula MAC si hay `DE64` o `DE128`
- `processResponse(req, resp)`:
- verifica ARQC
- puede sobreescribir `DE39`
- agrega `91` con ARPC
- agrega MAC respuesta
- Herramientas: `verifyArqc`, `verifyMac`, `decryptPin`, `key(name)`
- Claves solo vía `secrets/ENC(...)`; mostrar solo KCV.

sim-iso-parser

- Modelo canónico: `IsoMessage`.
- `MessageService`:
- registro y caché
- pack/unpack JSON/wire
- `convert(msg, targetDialect)`
- `responseHeader(dialect, reqHeader)`
- Codecs:
- `IsoCodec`
- `FieldCodec`
- `Tlv`
- `Base24Tokens`
- Dialectos built-in:
- `iso87-ascii`: ISO 8583:1987 ASCII base.
- `iso87-binary`: 1987 con numerics BCD y bitmap binario.
- `iso93-ascii`: ISO 8583:1993 ASCII; cambia varios campos.
- `base24-pos`: BASE24 POS external message; header ISO, bitmap hex ASCII, fields privados 52/60/61/63/64/128.
- `base24-atm`: variante ATM con header product indicator 01 y S-126.
- `visa-base1`: VISA BASE I/V.I.P., header binario 22 bytes y length embebido.
- `mastercard-cis`: Mastercard CIS/Banknet, EBCDIC y DE48/55/61 específicos.
- `hpdh`: Hypercom POS/TPDU, TPDU 5 bytes, BCD MTI/numerics.
- `bnet`: plantilla BNET editable, ASCII data, hex bitmap, length ASCII.

sim-validator

- `validateStructure(target, msg, dialect)`: campos definidos, tipo, largo, Luhn DE2.
- `validateRequest(profile, req, dialect)`.
- `validateExchange(profile, req, resp, dialect)`: MTI respuesta, eco, RC esperado, reglas.
- `compare(expected, actual, ignoreFields)`.
- `ValidationProfile` con `expectedRc`, `echoFields` por defecto `2,3,4,11,37,41`, `requestRules`, `responseRules`.
- `FieldRule` tipos: `REQUIRED`, `ABSENT`, `EQUALS`, `NOT_EQUALS`, `REGEX`, `IN`, `LENGTH`, `NUMERIC`, `LUHN`, `RANGE`, `ECHO`, `STARTS_WITH`.

sim-persistence

- Datos en `~/.simulator/data` o `-Dsimulator.data=`.
- Migrations:
- V1 crea `test_suite`, `test_case`, `transaction_log`, `regression_run`, `regression_result`, `stress_run`, `stress_sample`, `base24_snapshot`, `audit_log`.
- V2 agrega `app_user`, `approval_request`.
- V3 elimina usuarios/aprobaciones.
- V4 renombra `audit_log` a `system_audit_log`, `regression_*` y `stress_*` a `*_execution`, crea `dialect`, `unit_test`, `regresion`, `stress_test`, migra datos, crea nuevo `audit_log`.
- V5 inserta casos de ejemplo en `unit_test`, `regresion`, `stress_test` y `audit_log`.
- V6 elimina `validation_json`.
- Tablas finales tras V1..V6:
- `system_audit_log` (de V4 renombrada desde `audit_log`): auditoría operativa previa.
- `regression_execution`, `regression_execution_result`, `stress_execution`, `stress_execution_sample`: historial operacional antiguo.
- `dialect(id, description, created_at, updated_at)`
- `unit_test(id, suite_id, name, request, expected_rc, ignore_fields, enabled, created_at, updated_at)`
- `regresion(id, suite_id, name, request, expected_rc, ignore_fields, enabled, created_at, updated_at)`
- `stress_test(id, suite_id, name, request, expected_rc, ignore_fields, enabled, created_at, updated_at)`
- `audit_log(id, request, response, created_at, status)`
- Nota: persisten tablas viejas renombradas por V4, además de las nuevas.

sim-app

- `Launcher`: con `--headless` delega en `Headless.run`.
- `Headless`:
- `regression --suite [--env] [--concurrency] [--junit] [--html]`
- `issuer [--port] [--env]`
- `agent --token [--port] [--bind] [--name] [--env]`
- opción común `--metrics` expone `/metrics`
- lee `SIM_MASTER_PASSWORD`
- props: `-Dsimulator.config`, `-Dsimulator.data`, `-Dsimulator.logs`
- `build.bat [skiptests] [offline]`
- `run.bat [args]`

sim-common

- Uso transversal.
- Regla clave: todo lo logueado o persistido debe pasar por `Masking.maskFields`.
- `TransactionEvent.source` usa strings libres; valores típicos: `ACQUIRER`, `ISSUER`, `ISSUER_CLIENT`, `REGRESSION`, `STRESS`, `REPLAY`, `API`.

Si quieres, en el siguiente paso puedo convertir esto en una tabla manual final en español, lista para pegar en tu documentación, con formato más editorial.

Apéndice C. Evidencia y archivos de referencia

Recurso	Ubicación / propósito
README principal	README.md — visión general, primer uso, configuración y changelog.
Memoria técnica	skills/<módulo>/SKILL.md — clases, invariantes, tests y extensión.
Defaults	sim-config/src/main/resources/defaults/*.json.
Reglas emisor	sim-issuer/src/main/resources/issuer/default-rules.json.
Formatos	sim-iso-parser/src/main/resources/formats/ y config/formats/.
Migraciones	sim-persistence/src/main/resources/db/migration/V*.sql.
Prometheus/Grafana	docs/prometheus.yml y docs/grafana-dashboard.json.
JAR	sim-app/target/simulator-payment.jar.

Comandos de verificación

```
mvn -B -pl sim-app -am package
java -jar sim-app\target\simulator-payment.jar

# Test focalizado de transporte
mvn -B -pl sim-transport -am test

# Test focalizado del emisor
mvn -B -pl sim-issuer -am test
```

Fin del manual. Este documento refleja el comportamiento del código y la interfaz disponibles al 30/09/2026. Ante una modificación funcional, actualice README, SKILLs y este manual.